

Frontend Advanced

Next.js 기초와 렌더링

Next.js 기초

Next.js 개요



NEXT.JS

Next.js는 높은 퀄리티의 풀스택 웹 애플리케이션을 제작할 수 있는
React 기반의 프레임워크이다.



프레임워크란?

Framework

Next.js에 대해 알아보기 전에, 프레임워크가 무엇인지 알아보자.



개발자들이 현업에서 대규모의 프로그램을 개발해야 한다.

따라서 프로그램의 전체 흐름을 한 줄씩 모두 구현하는 것에는 한계가 있다.



Framework

Library

이를 해결하기 위해 개발자는 프레임워크와 라이브러리를 사용한다.



Framework

Library

```
npm i next
```

```
npm i react react-dom
```

프레임워크와 라이브러리 둘 다 이미 만들어진 코드의 모음이라는 부분에서 동일하다.

따라서 Node.js에서 둘 모두 npm을 통해 설치를 진행한다.



Framework



Library



이 둘의 가장 큰 차이점은 바로 “코드 흐름 제어의 주도권을 누가 가지고 있는지”이다.



Library



라이브러리는 코드의 흐름을 개발자가 주로 제어한다.

따라서 라이브러리는 개발에 도움이 되는 클래스, 함수 등을 제공하고,

이를 적절히 사용하는 것은 개발자의 몫이다.



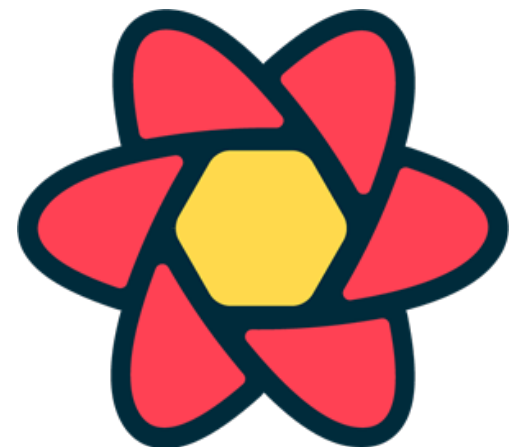
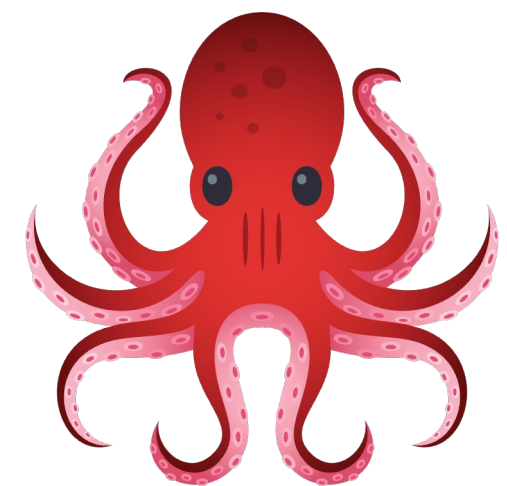
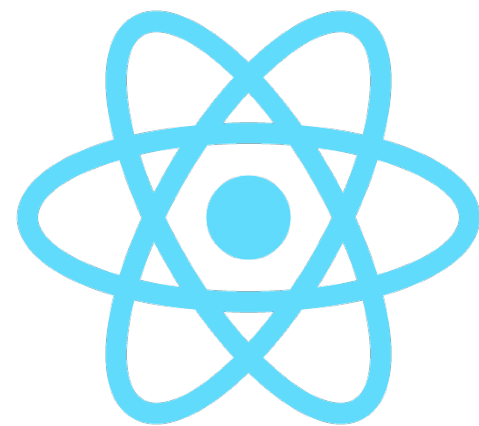
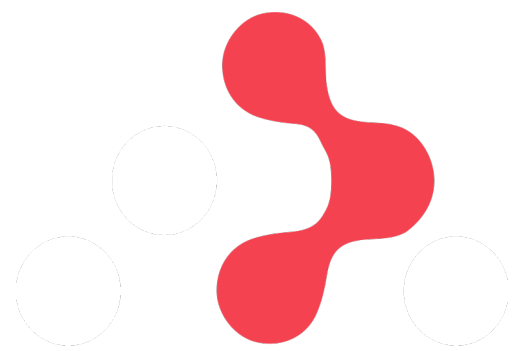
Framework



반면 프레임워크는 프레임워크가 코드의 흐름을 제어한다.

따라서 정해진 규칙대로만 개발자가 코드를 작성하면, 애플리케이션은 적절하게 실행된다.

Library



지금까지 사용해온 기술들은 대부분 라이브러리이다.

따라서 우리(개발자)가 작성한 코드 흐름에서 새로운 기능을 추가하기 위해 라이브러리를 추가로 설치했다.



Framework

NEXT.js



django

반면 프레임워크를 사용하게 되면, 전체 애플리케이션의 흐름이 프레임워크에 이미 구성되어 있고, 추가적인 라이브러리 설치를 하지 않아도 대부분의 기능이 프레임워크에 포함되어 있다.

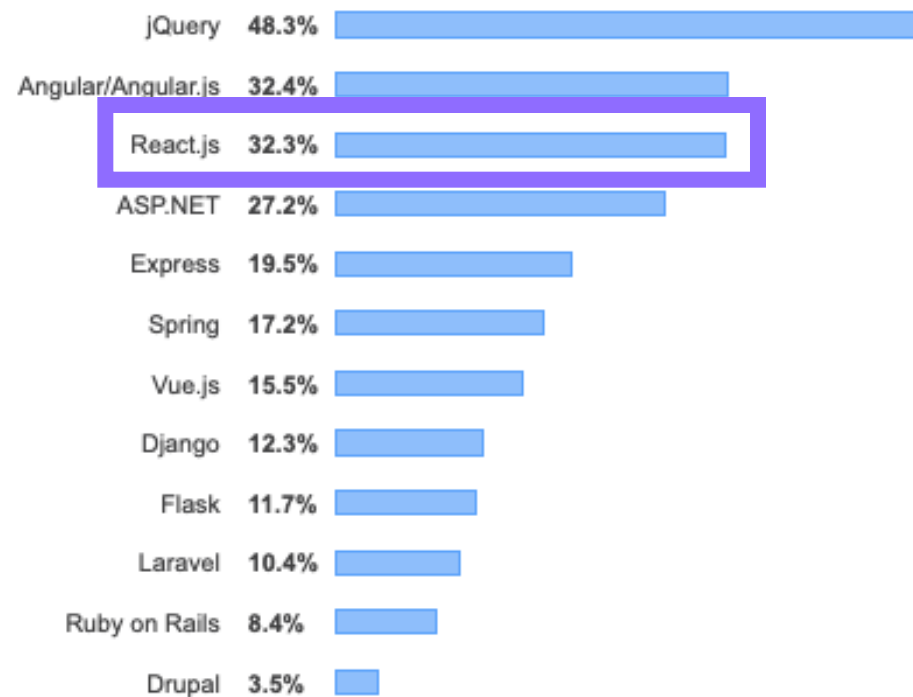


React와 프레임워크

Web Frameworks

All Respondents

Professional Developers



55,079 responses; select all that apply

This year, we asked about frameworks for the web separately from other frameworks and libraries. jQuery is the most broadly used of these web frameworks, and this year more developers say they use React.js than Angular, a switch from last year.

2019

Web frameworks and technologies

The +5 point increase for FastAPI is one of the most significant shifts in the web framework space. This signals a strong trend towards using Python for building performant APIs and reflects the overall strength of the Python ecosystem.

Which web frameworks and web technologies have you done extensive development work in over the past year, and which do you want to work in over the next year? (If you both worked with the framework and want to continue to do so, please check both boxes in that row.)

2025

All Respondents

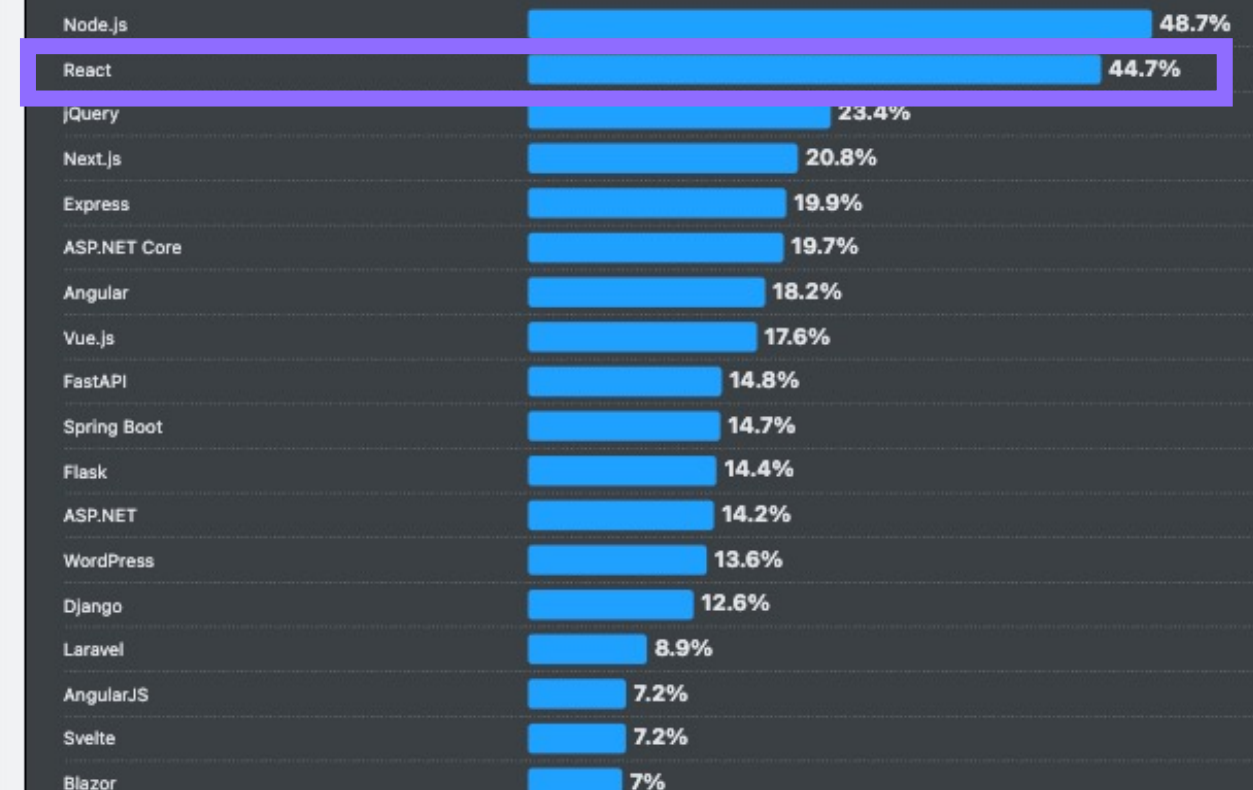
Professional Developers

Learning to Code

Professionals that Use AI

Learners that Use AI

Write-Ins



React는 계속 발전하고 있고, 현재 대부분의 웹 프론트엔드 애플리케이션의 기반이 되고 있다.

→ 즉, React는 이제 웹 애플리케이션의 기본 사양이 되고 있다.

<https://survey.stackoverflow.co/2019#technology>

<https://survey.stackoverflow.co/2025/technology#most-popular-technologies-webframe-prof>

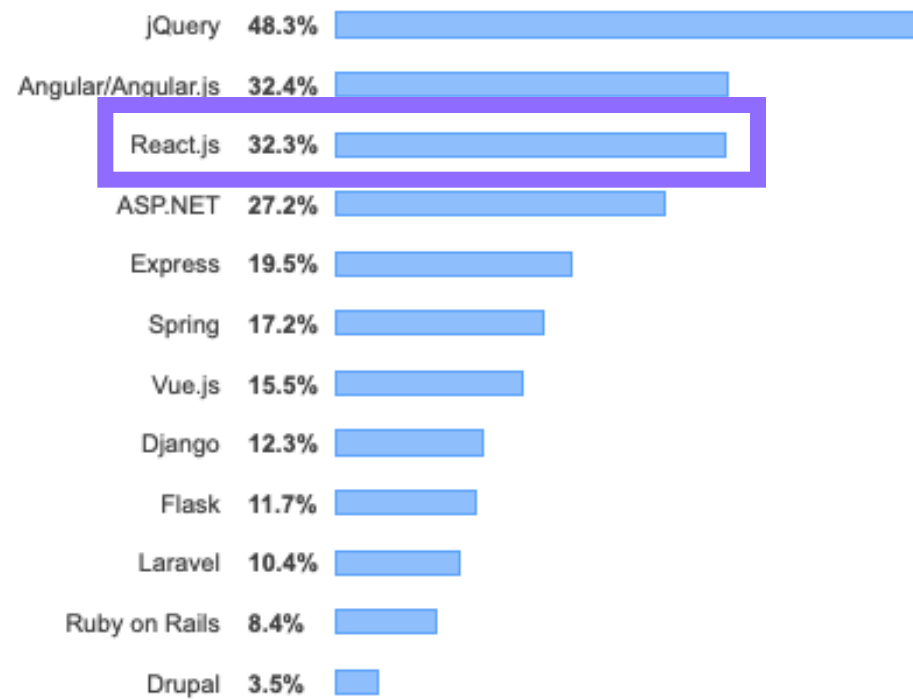


React와 프레임워크

Web Frameworks

All Respondents

Professional Developers



55,079 responses; select all that apply

This year, we asked about frameworks for the web separately from other frameworks and libraries. jQuery is the most broadly used of these web frameworks, and this year more developers say they use React.js than Angular, a switch from last year.

2019

Web frameworks and technologies

The +5 point increase for FastAPI is one of the most significant shifts in the web framework space. This signals a strong trend towards using Python for building performant APIs and reflects the overall strength of the Python ecosystem.

? Which web frameworks and web technologies have you done extensive development work in over the past year, and which do you want to work in over the next year? (If you both worked with the framework and want to continue to do so, please check both boxes in that row.)

2025

All Respondents

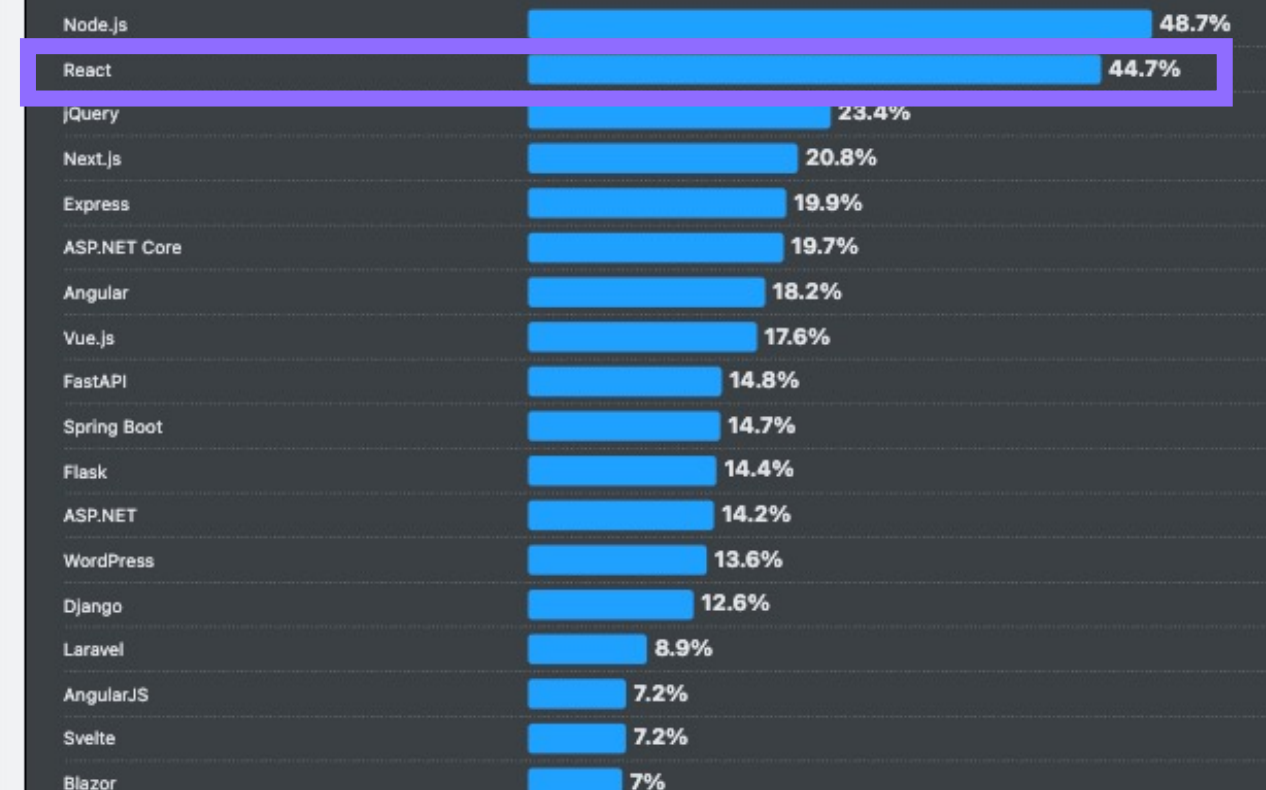
Professional Developers

Learning to Code

Professionals that Use AI

Learners that Use AI

Write-Ins



→ 즉, React는 이제 웹 애플리케이션의 기본 사양이 되고 있다.

= 웹 개발을 하기 위한 기본 언어처럼 여겨지고 있다.

<https://survey.stackoverflow.co/2019#technology>

<https://survey.stackoverflow.co/2025/technology#most-popular-technologies-webframe-prof>



DEEP DIVE

프레임워크 없이 React 를 사용할 수 있나요?

^ 숨기기

물론 React를 프레임워크 없이 사용할 수 있습니다. 기존 페이지의 일부분에 React 사용하기를 활용해 보세요. 하지만, 새로운 애플리케이션이나 사이트 전체를 React로 구축하는 경우에는 프레임워크 사용을 권장합니다.

이유가 여기 있습니다.

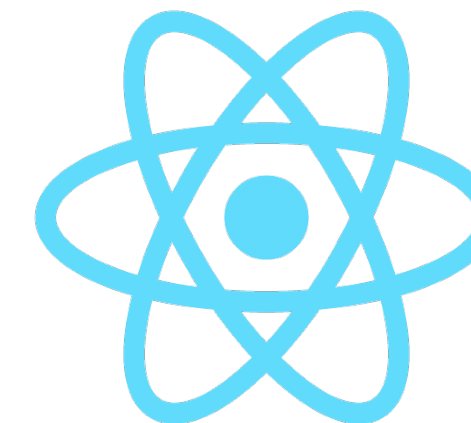
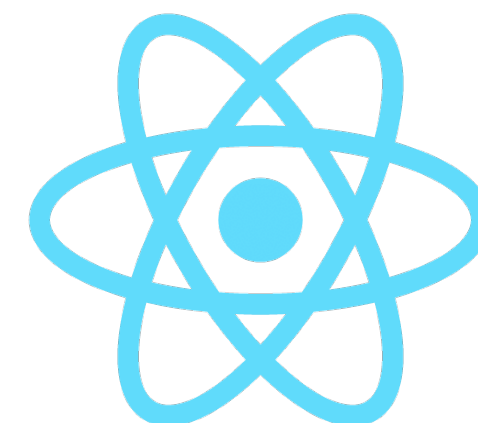
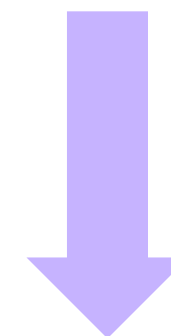
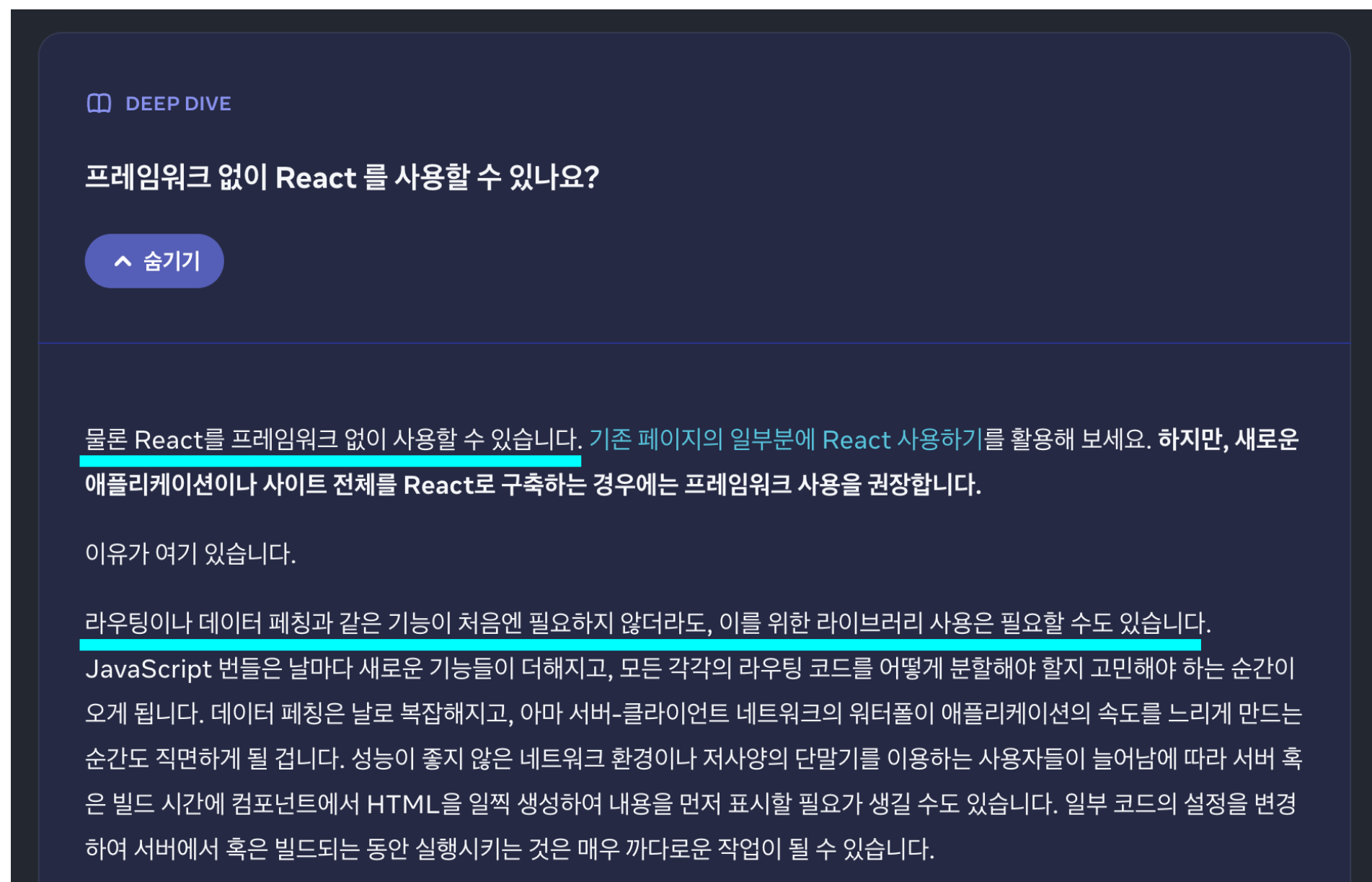
라우팅이나 데이터 페칭과 같은 기능이 처음엔 필요하지 않더라도, 이를 위한 라이브러리 사용은 필요할 수도 있습니다.

JavaScript 번들은 날마다 새로운 기능들이 더해지고, 모든 각각의 라우팅 코드를 어떻게 분할해야 할지 고민해야 하는 순간이 오게 됩니다. 데이터 페칭은 날로 복잡해지고, 아마 서버-클라이언트 네트워크의 워터폴이 애플리케이션의 속도를 느리게 만드는 순간도 직면하게 될 겁니다. 성능이 좋지 않은 네트워크 환경이나 저사양의 단말기를 이용하는 사용자들이 늘어남에 따라 서버 혹은 빌드 시간에 컴포넌트에서 HTML을 일찍 생성하여 내용을 먼저 표시할 필요가 생길 수도 있습니다. 일부 코드의 설정을 변경하여 서버에서 혹은 빌드되는 동안 실행시키는 것은 매우 까다로운 작업이 될 수 있습니다.

이러한 상황에서, React 팀은 React를 프레임워크와 함께 사용할 것을 권장하고 있다.



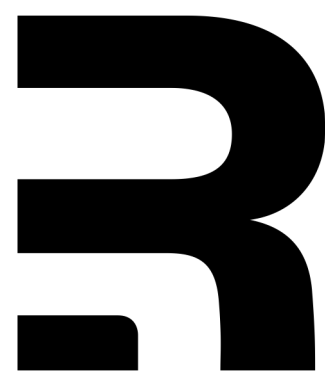
React와 프레임워크



HTML, CSS, JS라는 기본 언어를 더 잘 사용하기 위해 React를 사용하듯이,
이제는 **React 자체가 기본 언어가 되어**, 더 잘 사용하기 위해 React 기반 프레임워크를 사용하는 것이다.



Framework



NEXT.js



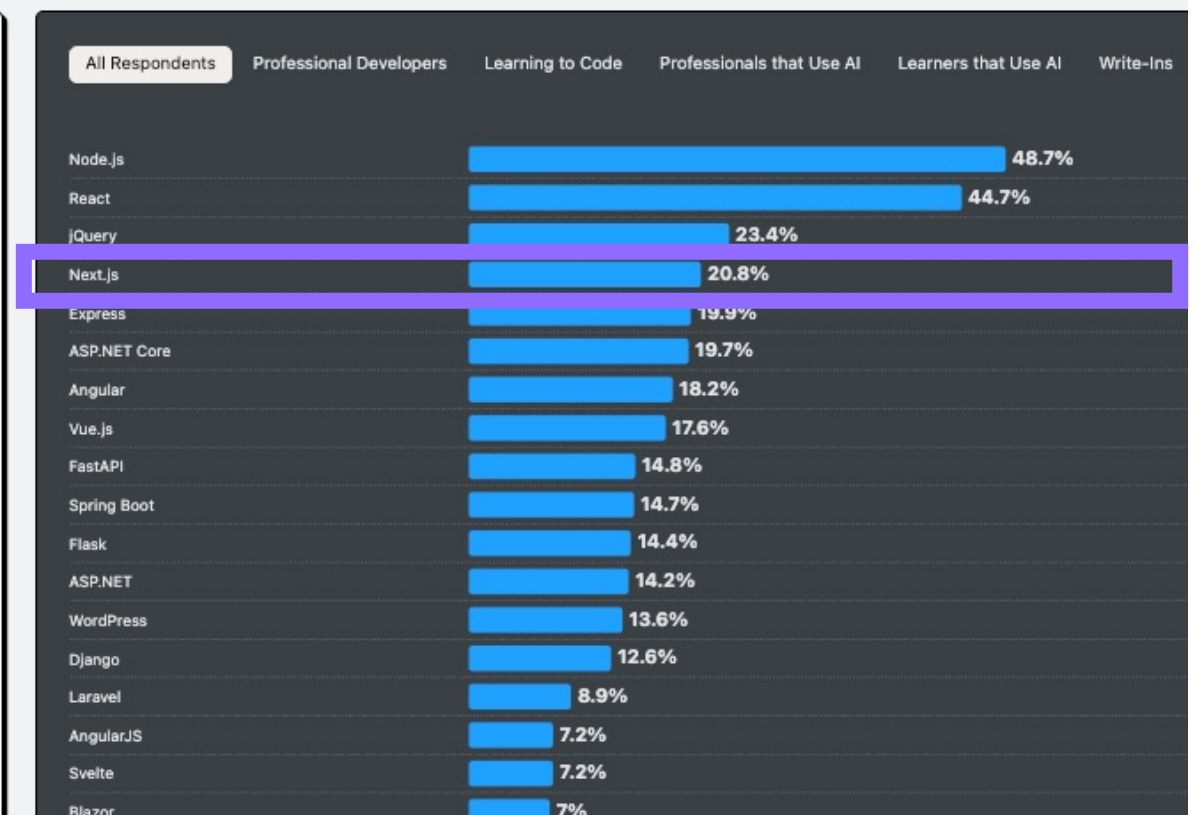
현업에서 많이 사용중인 React 기반 프레임워크로는
Remix, Next.js, Gatsby 등이 있다.

NEXT.JS

Web frameworks and technologies

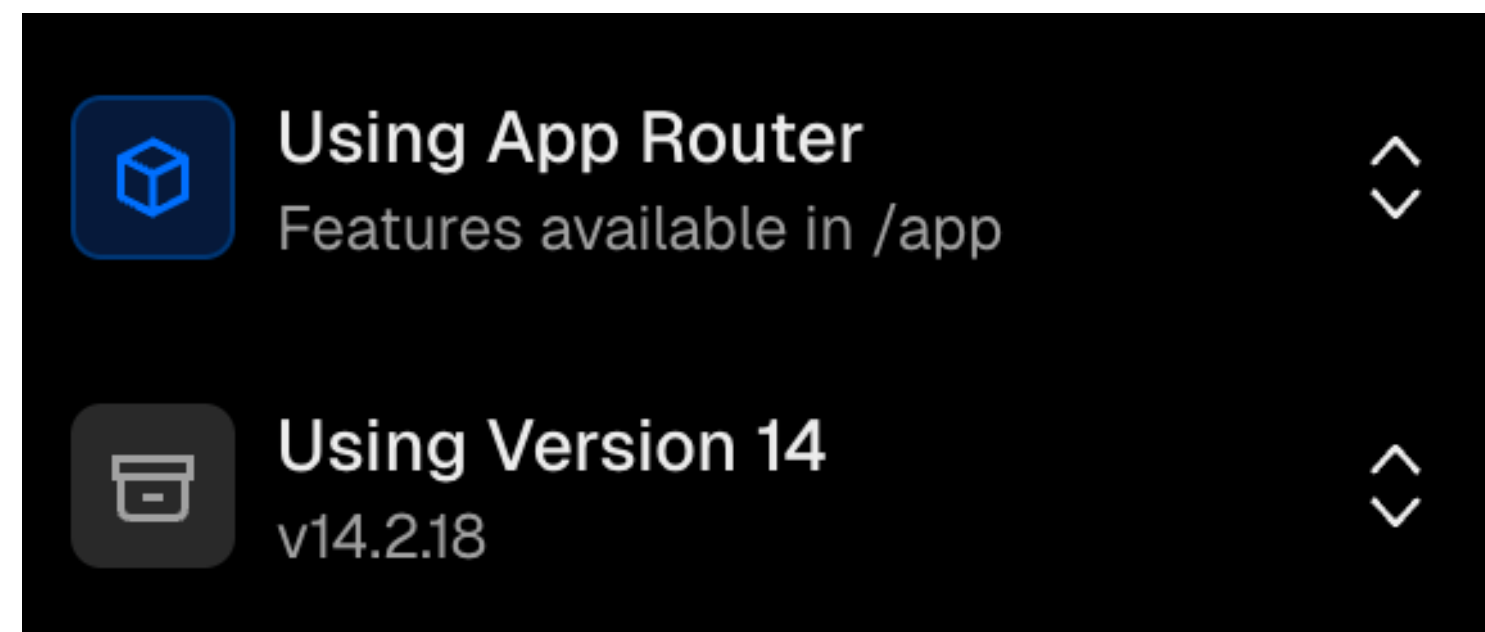
The +5 point increase for FastAPI is one of the most significant shifts in the web framework space. This signals a strong trend towards using Python for building performant APIs and reflects the overall strength of the Python ecosystem.

Which web frameworks and web technologies have you done extensive development work in over the past year, and which do you want to work in over the next year? (If you both worked with the framework and want to continue to do so, please check both boxes in that row.)



그 중에서도 가장 높은 점유율(20.8%*)을 가진 Next.js 프레임워크에 대해 학습해 보자.

NEXT.JS



Next.js 프레임워크는 버전 업데이트가 빈번히 이루어지는 프레임워크이다.

해당 챕터에서는 **Next.js 14(2023년 10월 27일)**를 기준으로 설명하며,
15(2024년 10월 22일), 16(2025년 10월 21일)에서 변경된 지점을 함께 언급할 예정이다.



create-next-app

```
npx create-next-app@14
```

React의 CRA와 같이 Next.js도 프로젝트 빠르게 시작하기 위한 npx 명령어가 존재한다.



```
iTerm
> npx create-next-app@14
Need to install the following packages:
create-next-app@14.2.18
Ok to proceed? (y)

✓ What is your project named? ... elice-next
✓ Would you like to use TypeScript? ... No / Yes
✓ Would you like to use ESLint? ... No / Yes
✓ Would you like to use Tailwind CSS? ... No / Yes
✓ Would you like to use `src/` directory? ... No / Yes
✓ Would you like to use App Router? (recommended) ... No / Yes
✓ Would you like to customize the default import alias (@/*)? ... No / Yes
Creating a new Next.js app in ~/Documents/elize-next.

Using npm.

Initializing project with template: app-tw

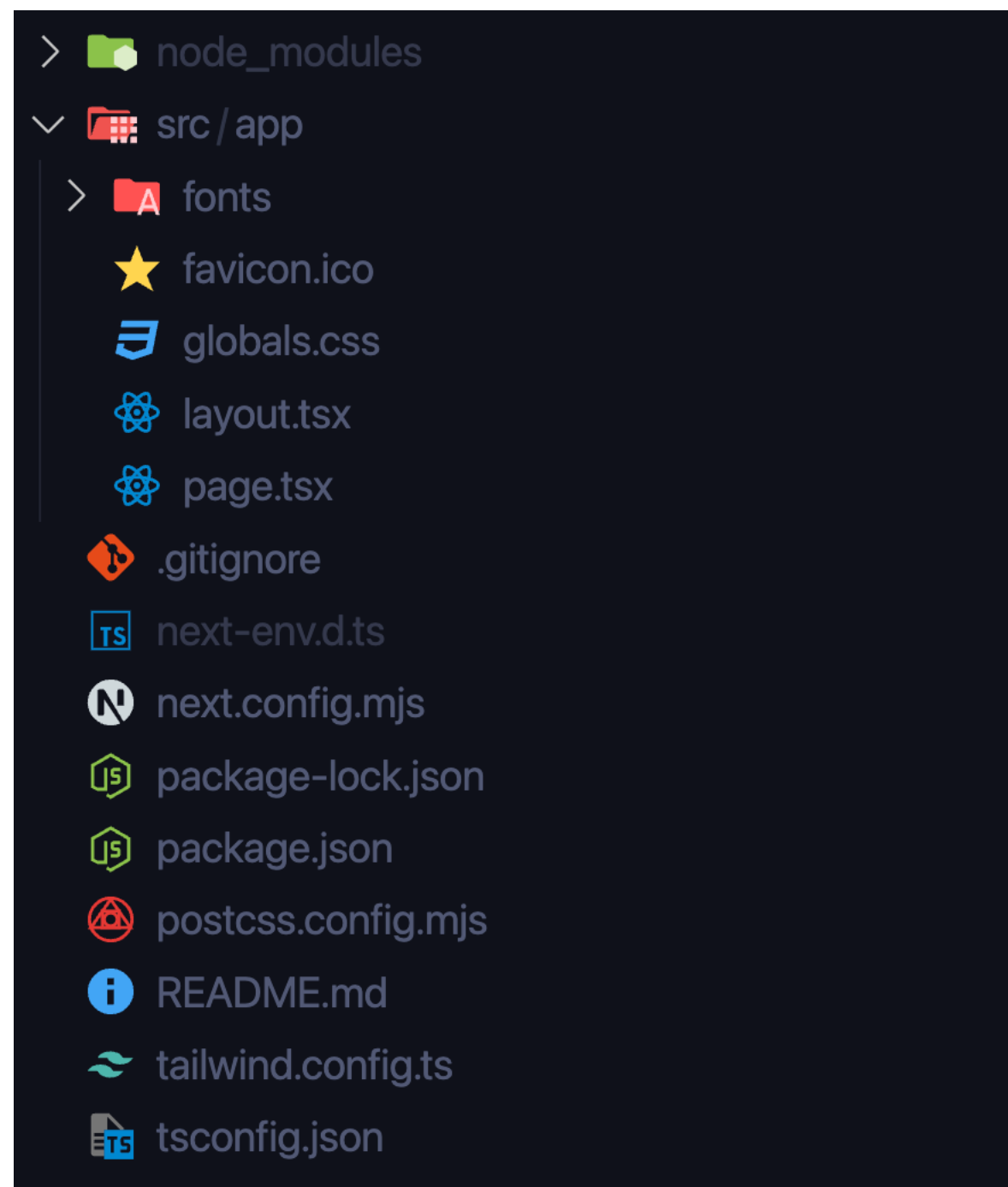
Installing dependencies:
- react
- react-dom
- next
```

위와 같이 설정을 진행하고, 원하는 에디터로 프로젝트 파일을 열어보자.

(해당 자료는 VSCode 기준)



프로젝트 구성

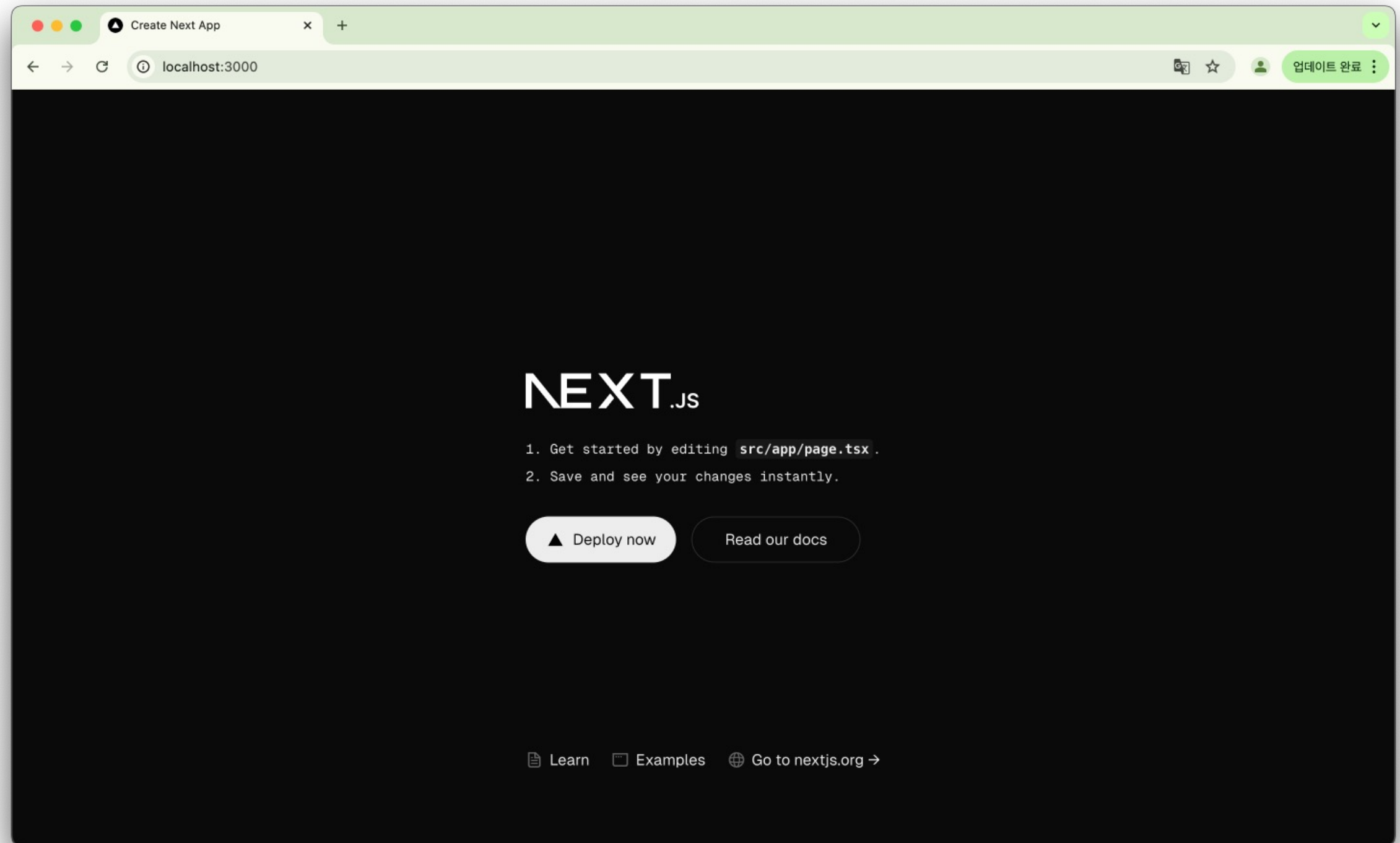


프로젝트가 위와 같이 구성되어 있다면 Next.js를 사용할 준비가 완료된 것이다.



프로젝트 구성

```
npm run dev
```



‘npm run dev’ 명령어를 통해 Next.js 개발 서버를 실행할 수 있다.



Routing과 Rendering

Data Fetching

Caching

Optimizing

프레임워크를 잘 사용하려면, 프레임워크의 흐름을 관통하는 핵심 개념을 이해해야 한다.

Next.js의 핵심 개념을 크게 4가지로 나누어보면 위와 같다.

Routing



Routing과 Rendering

Routing과 Rendering

Data Fetching

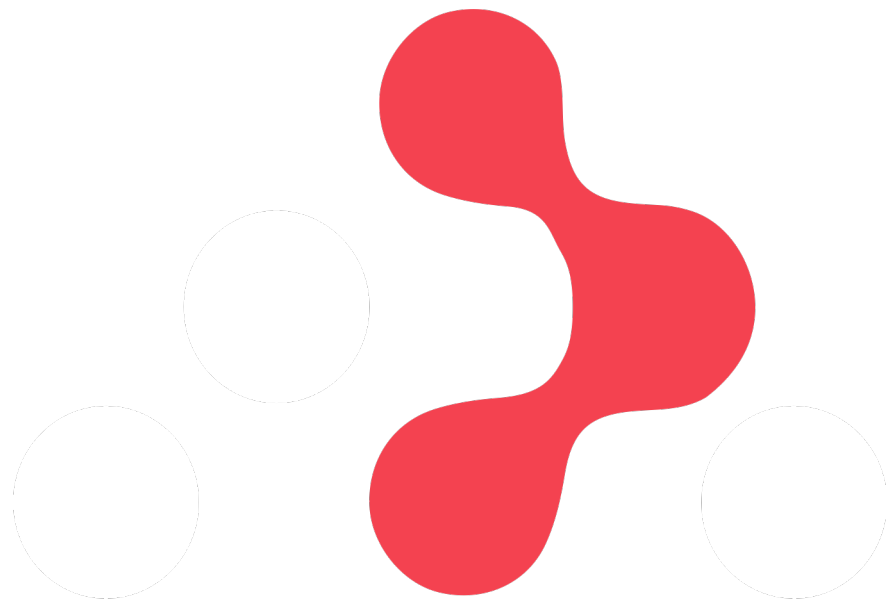
Caching

Optimizing

먼저 Next.js의 Routing과 Rendering 방식에 대해 알아보자.



기존의 Routing 방식



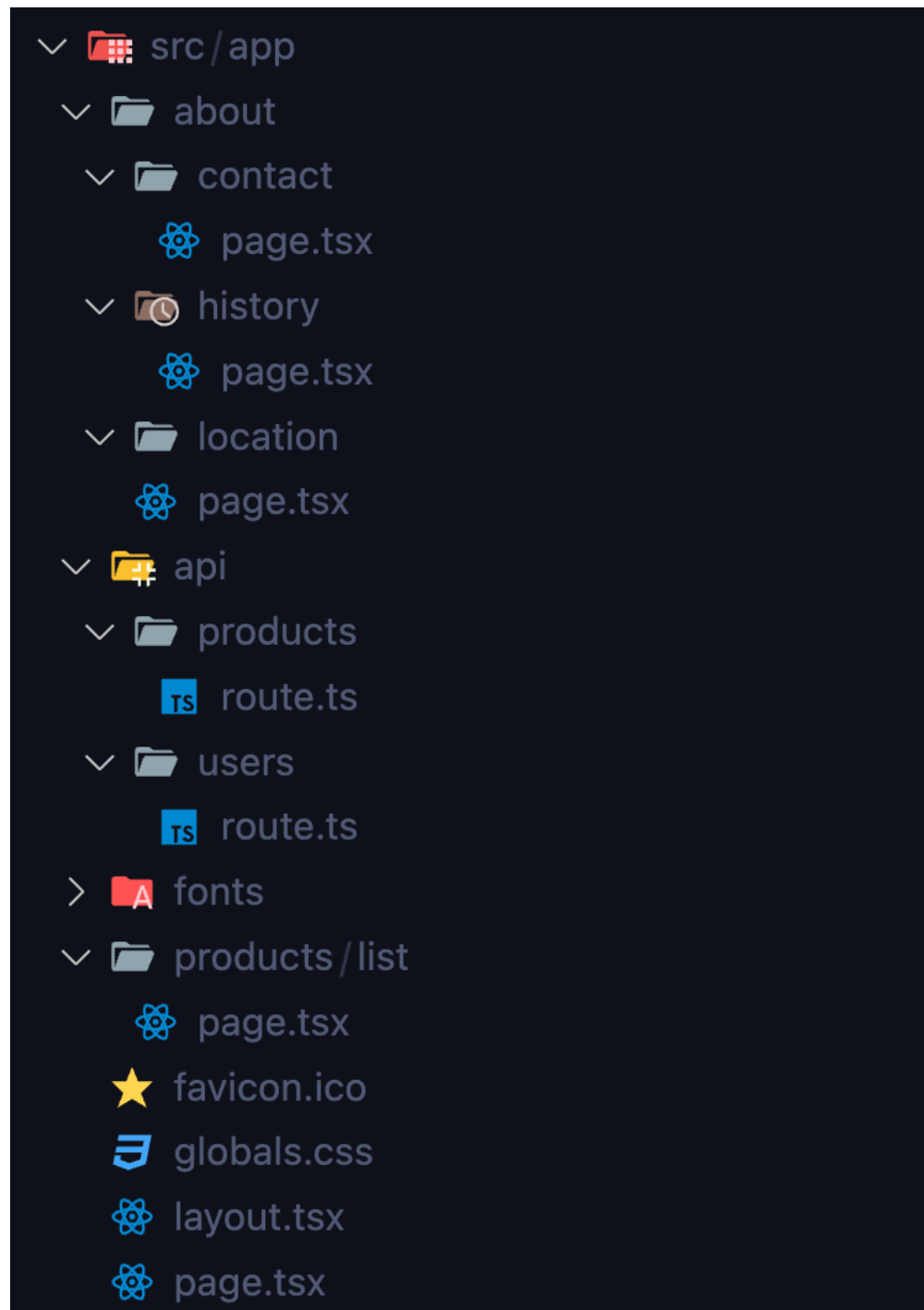
```
const router = createBrowserRouter([
  {
    path: "/",
    element: (
      <>
        <Nav />
        <Outlet />
      </>
    ),
    children: [
      { path: "", element: <Home /> },
      { path: "about", element: <About /> },
      { path: "contact", element: <Contact /> },
    ],
  },
]);

function App() {
  return <RouterProvider router={router} />;
}
```

React는 라우팅 기능을 내장하지 않아, 페이지 전환을 직접 구현하거나
외부 라이브러리(react-router-dom)를 사용해야 했다.



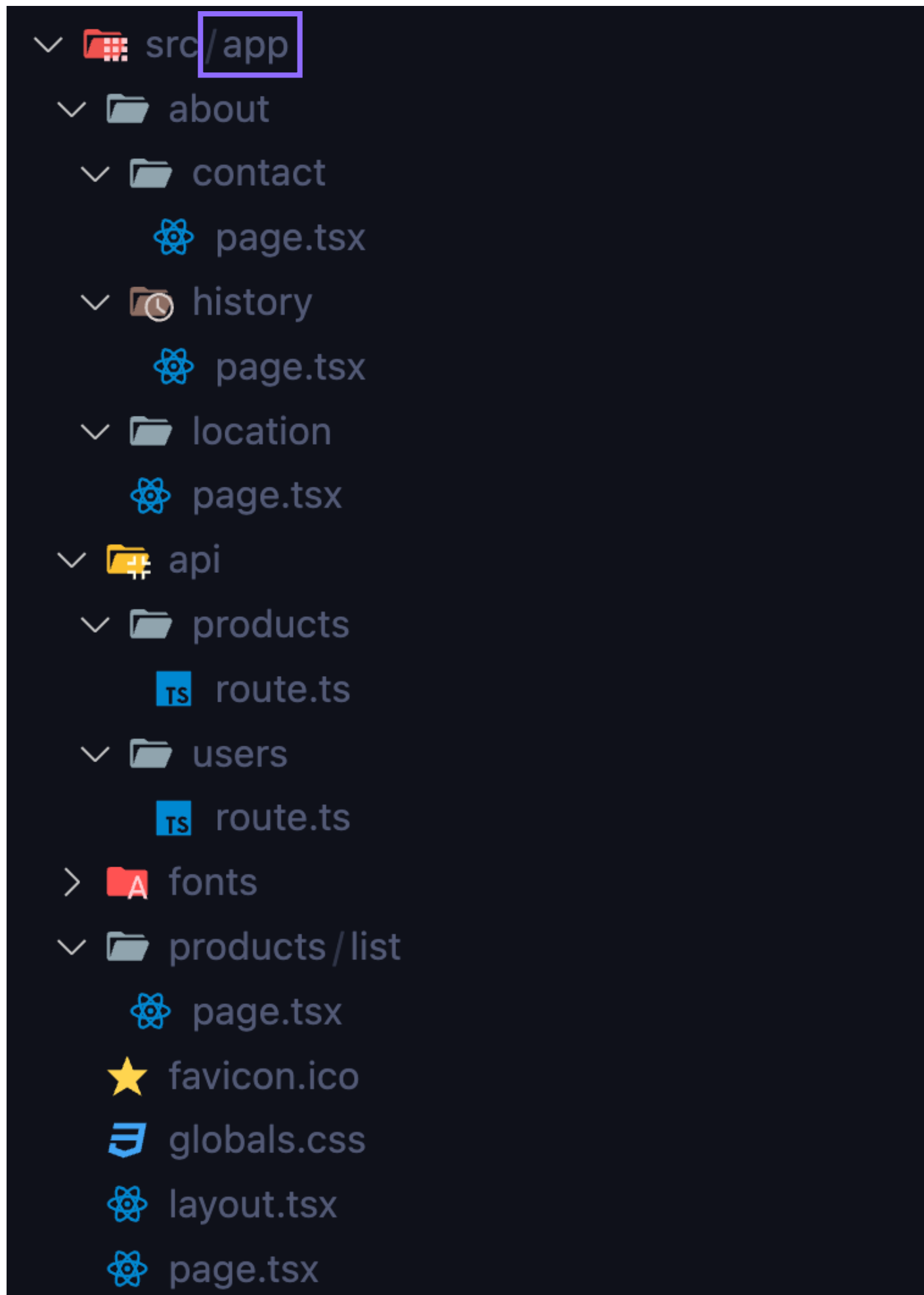
Next.js의 Routing 방식



Next.js에는 Routing 기능이 포함되어 있으며, File System을 기반으로 Routing을 수행한다.
파일과 디렉토리 구조를 정해진 규칙대로 구성하면, 이에 맞게 자동으로 Routing이 구현된다.



Routing 규칙 1

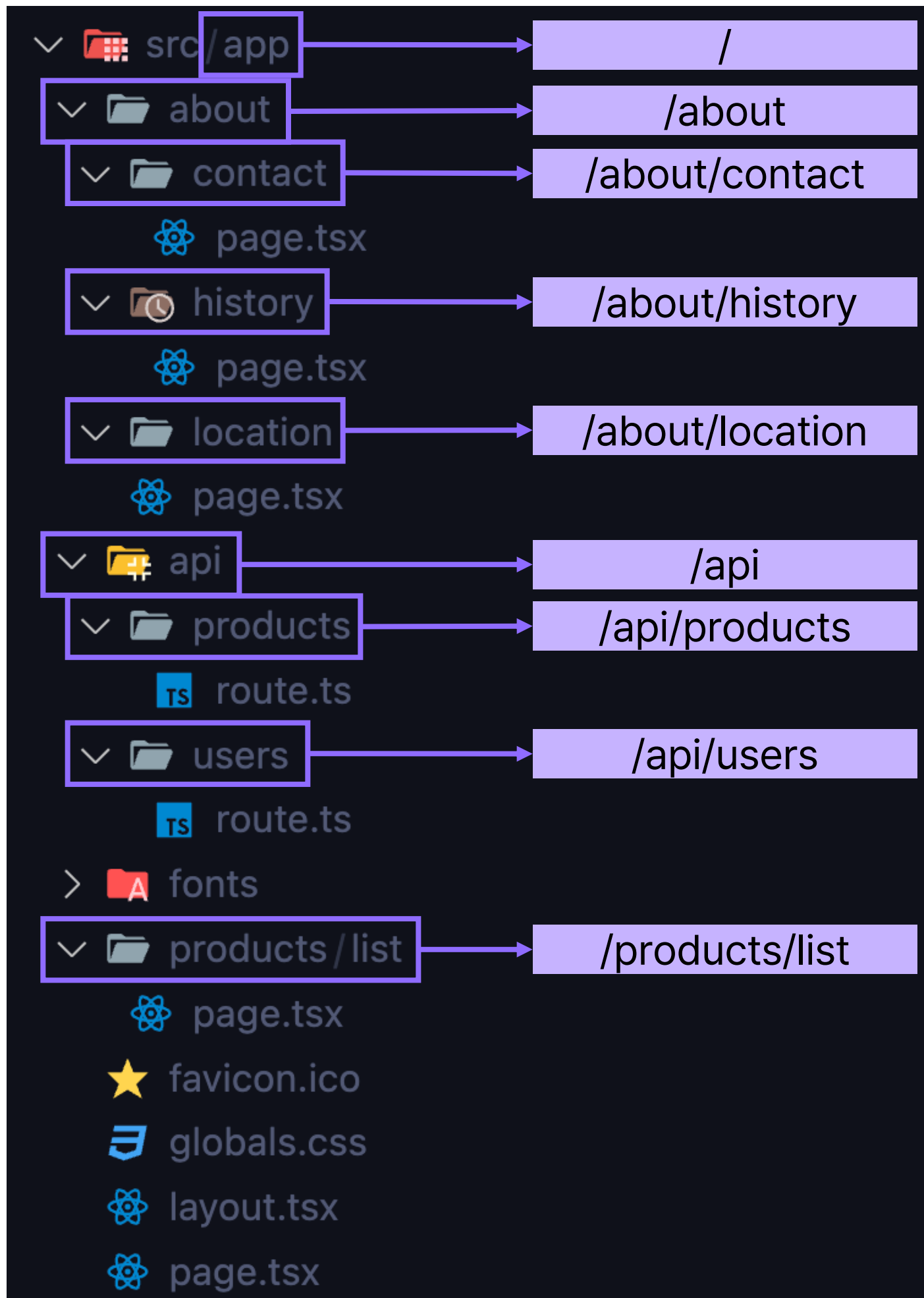


규칙 1.

Routing과 관련된 디렉토리와 파일은 모두 ``app/`` 디렉토리 하위에 위치한다.



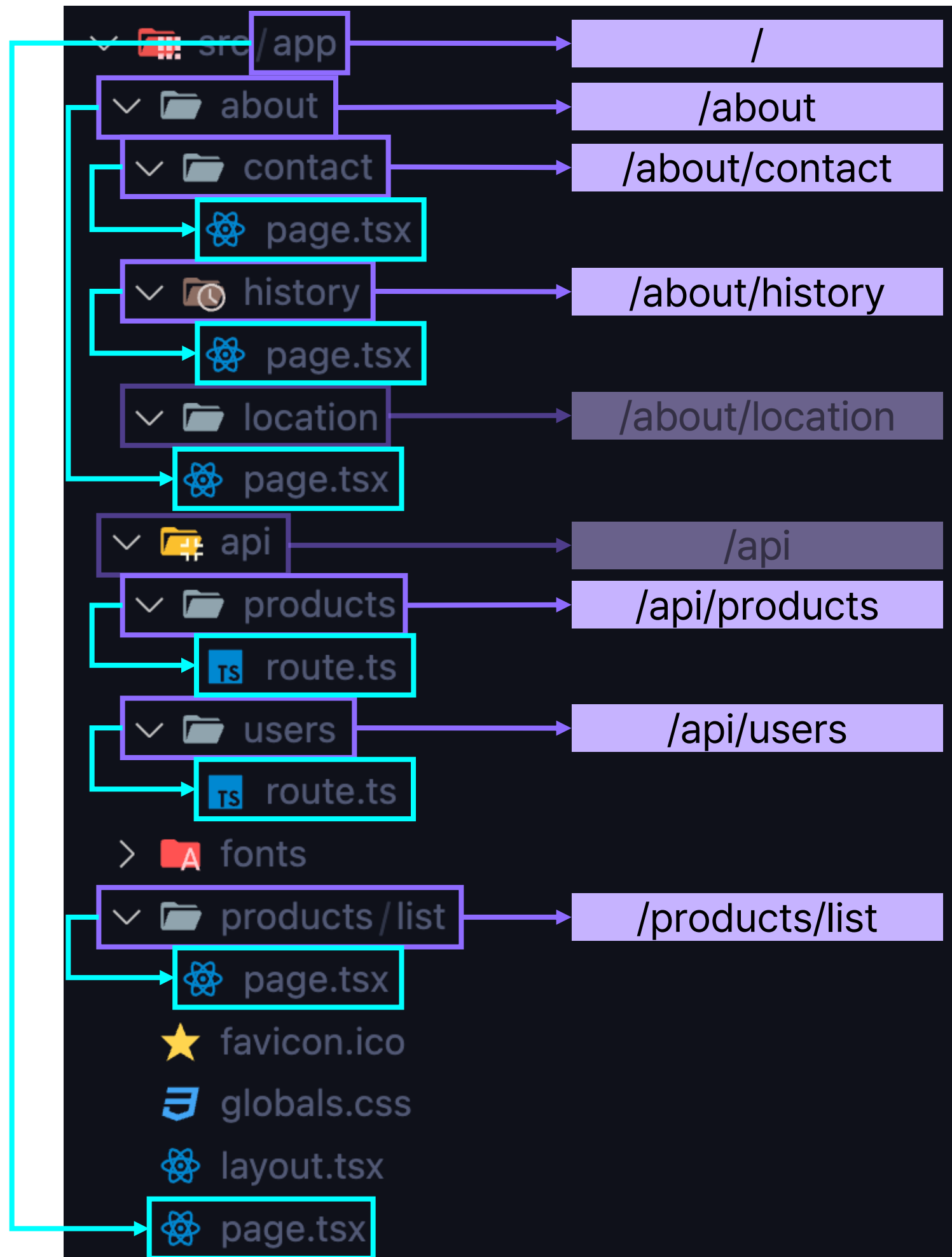
Routing 규칙 2



규칙 2.

각 **디렉토리**의 이름은 **URL의 path**에 대응된다.

이때, 최상위의 `app/` 디렉토리는 `/`에 대응된다.



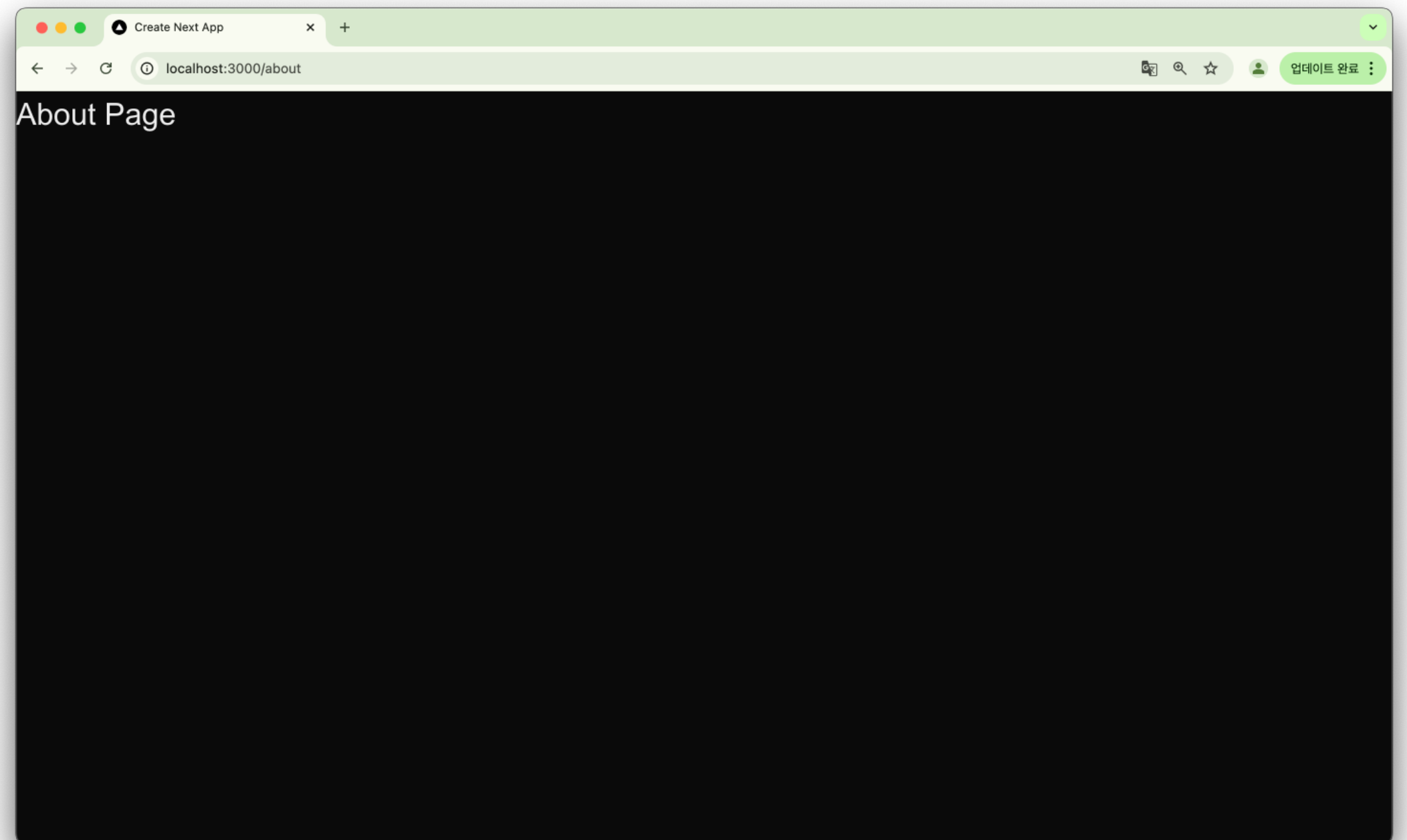
규칙 3.

디렉토리에 해당하는 path가 실제로 의미를 가지려면,
약속된 파일인 **page.tsx** 혹은 **route.ts** 파일이 존재해야 한다.

page.tsx (.js, .jsx도 가능)	해당 경로 접속 시 렌더링 할 UI 정의
route.ts (.js도 가능)	해당 경로에 대한 백엔드 API 정의 (추후 학습)

app/about/page.tsx

```
export default function () {  
  return <h1>About Page</h1>;  
}
```



‘page.tsx’ 파일에는 해당 path에 렌더링 될 컴포넌트를 정의하고,
export default로 보내는 코드를 작성한다.

app/about/page.tsx

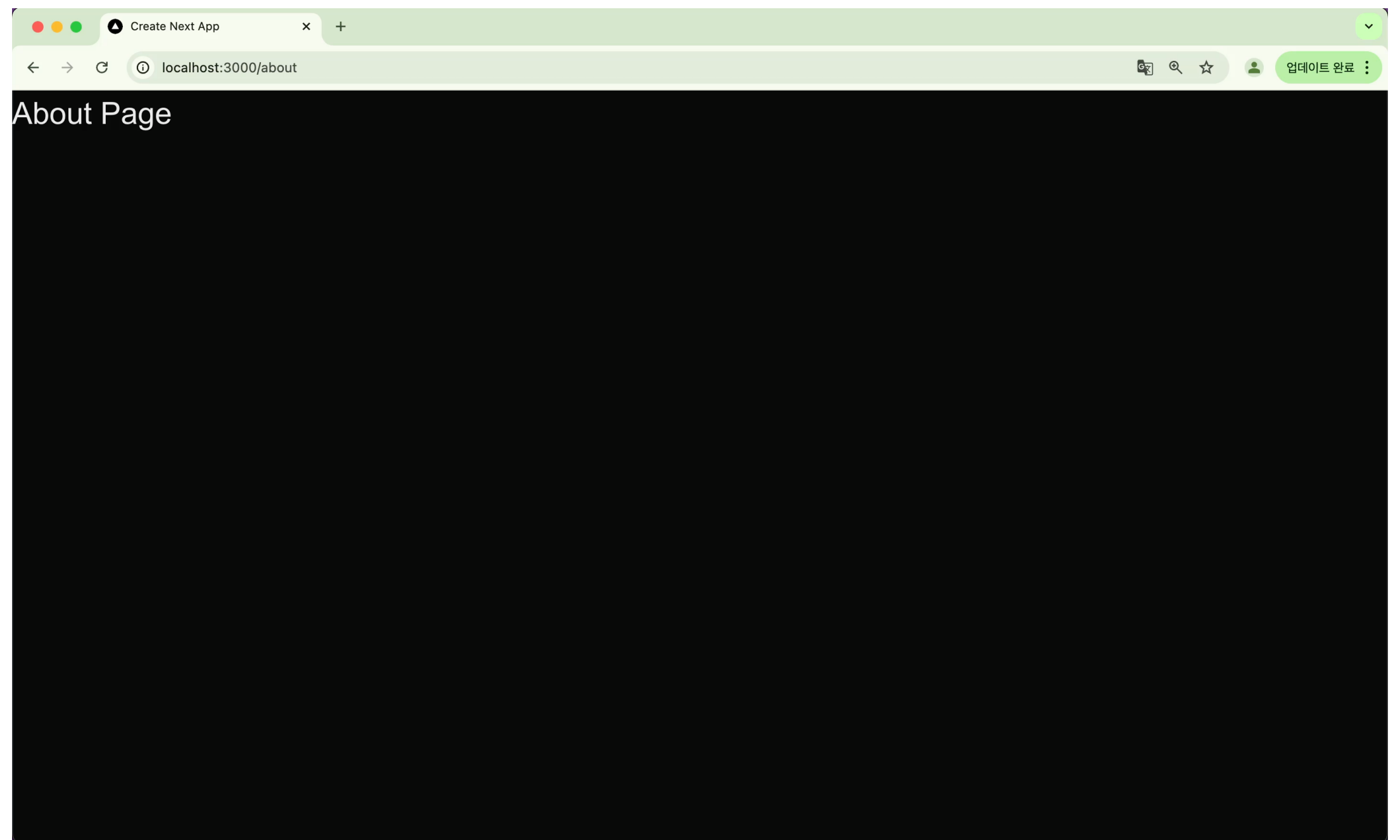
```
export default function () {  
  return <h1>About Page</h1>;  
}
```

app/about/contact/page.tsx

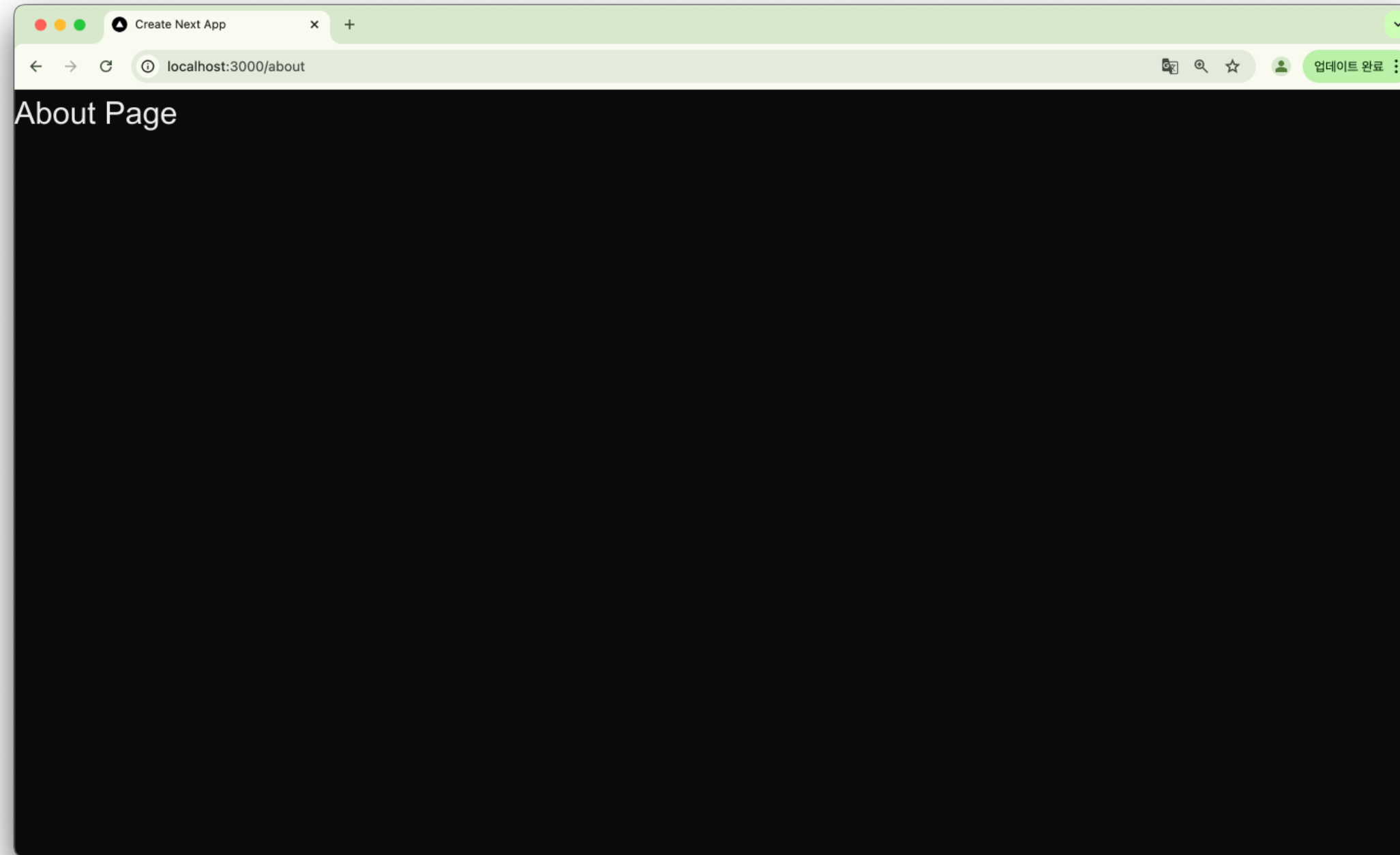
```
export default function () {  
  return <h1>Contact Page</h1>;  
}
```

app/products/list/page.tsx

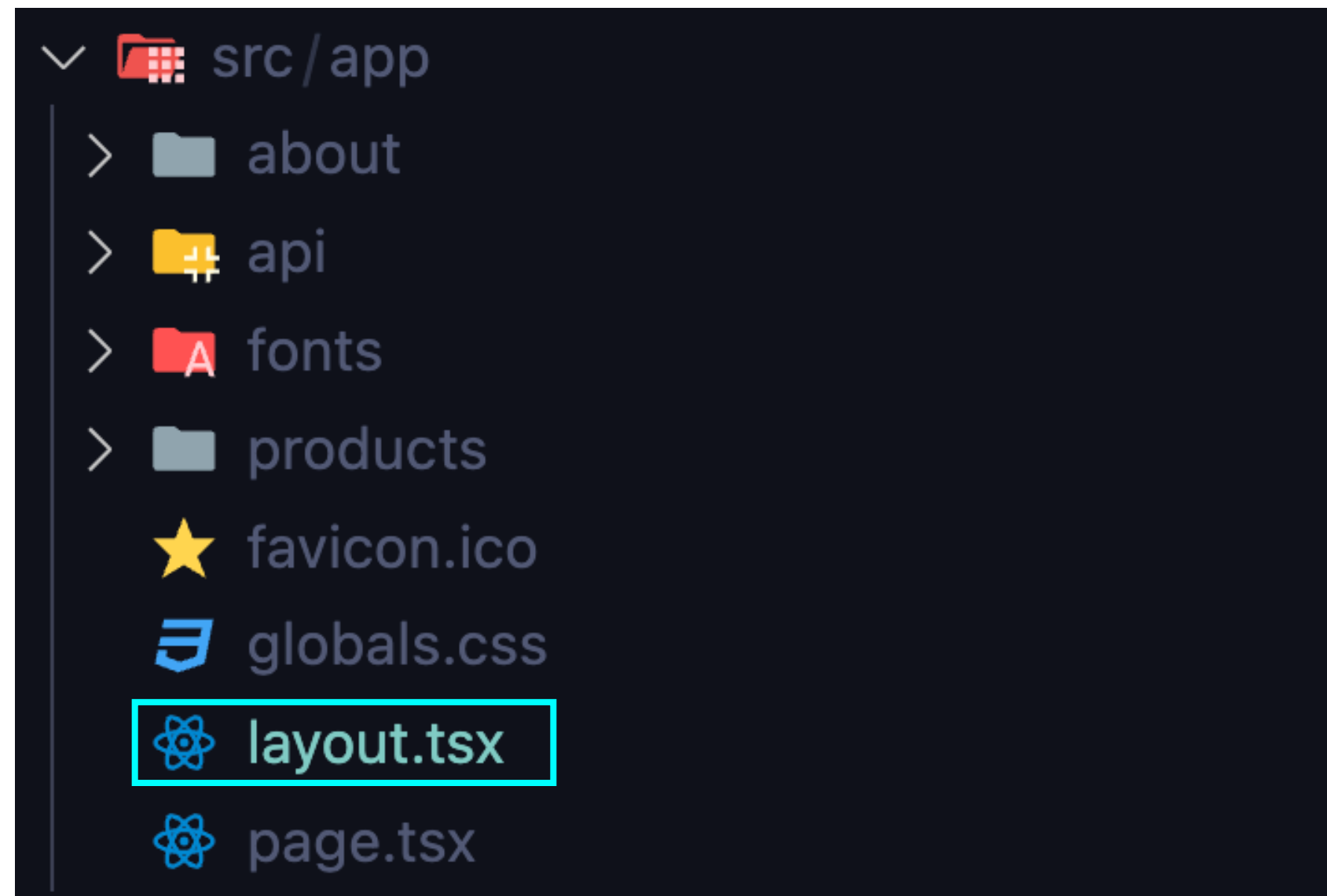
```
export default function () {  
  return <h1>Products List Page</h1>;  
}
```



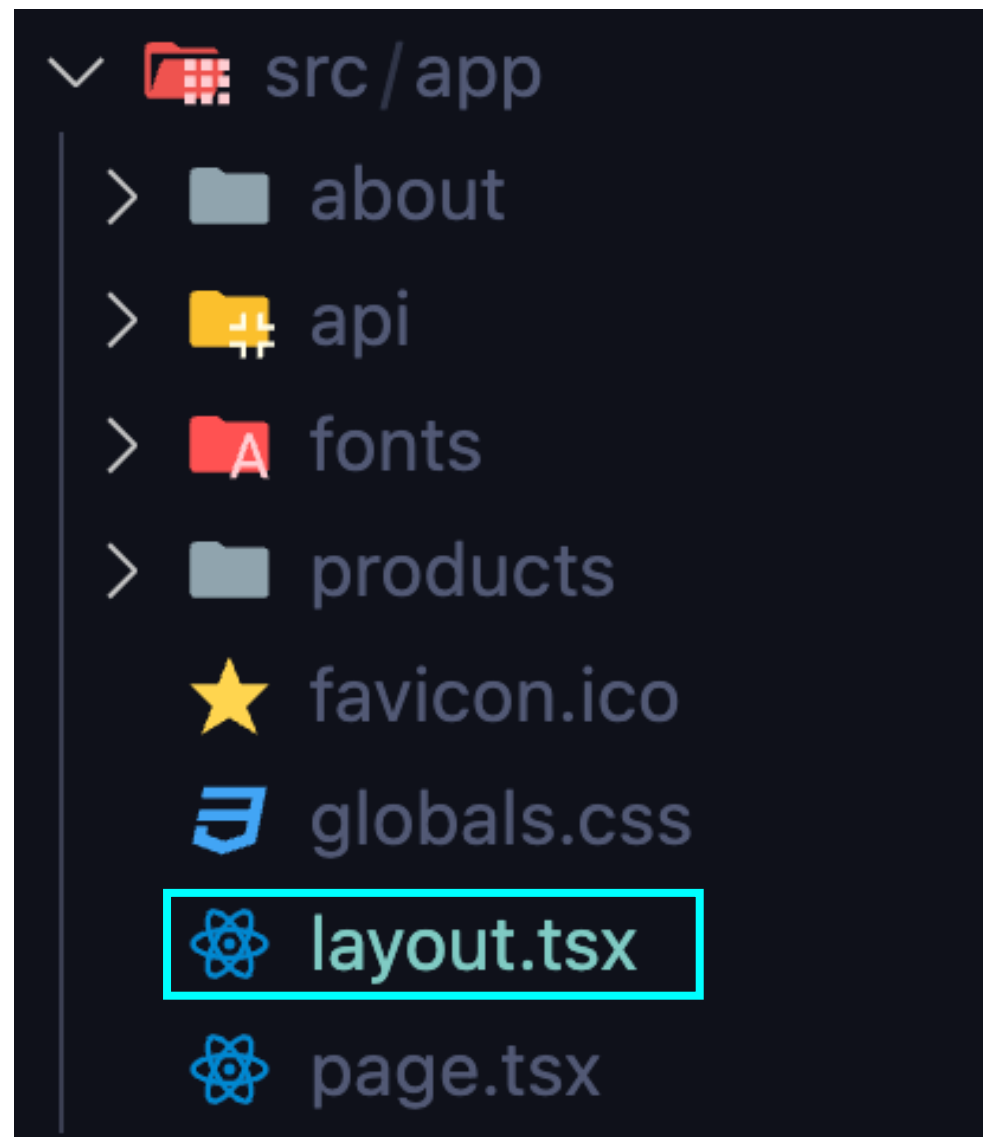
↑
Play



create-next-app으로 프로젝트 생성 후 바로 페이지를 작성하면,
위와 같이 검은 배경과 흰 글씨로 스타일이 적용된다.



그 이유는 ``app/`` 디렉토리에 위치한 ``layout.tsx`` 때문이다.



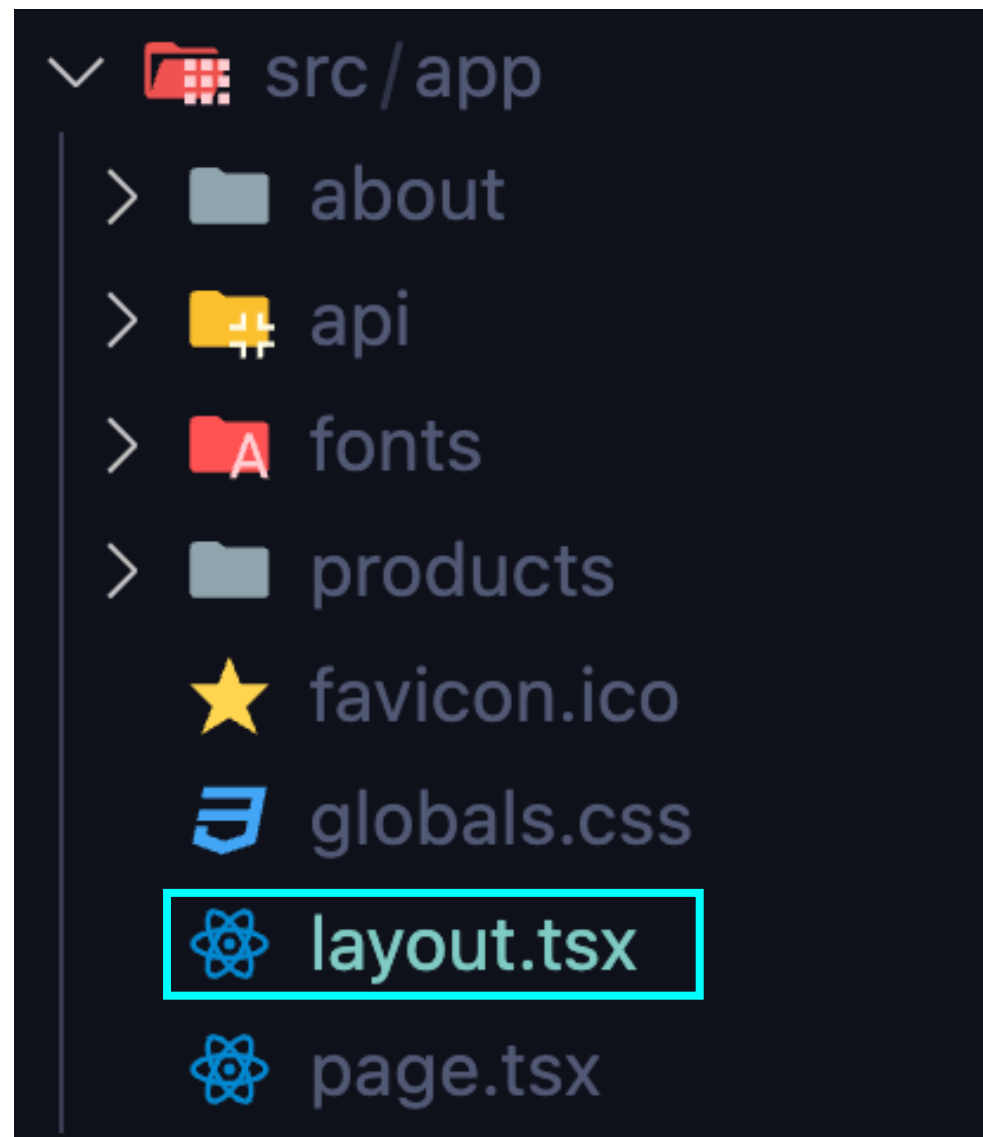
app/layout.tsx

```
import type { Metadata } from "next";
import localFont from "next/font/local";
import "./globals.css";

// ...생략

export default function RootLayout({ children }: Readonly<{ children: React.ReactNode }>){
  return (
    <html lang="en">
      <body className={`\${geistSans.variable} \${geistMono.variable} antialiased`} >
        {children}
      </body>
    </html>
  );
}
```

‘layout.tsx’는 또 다른 약속된 파일로, 하위 라우트와 공유하는 UI를 작성하는 파일이다.



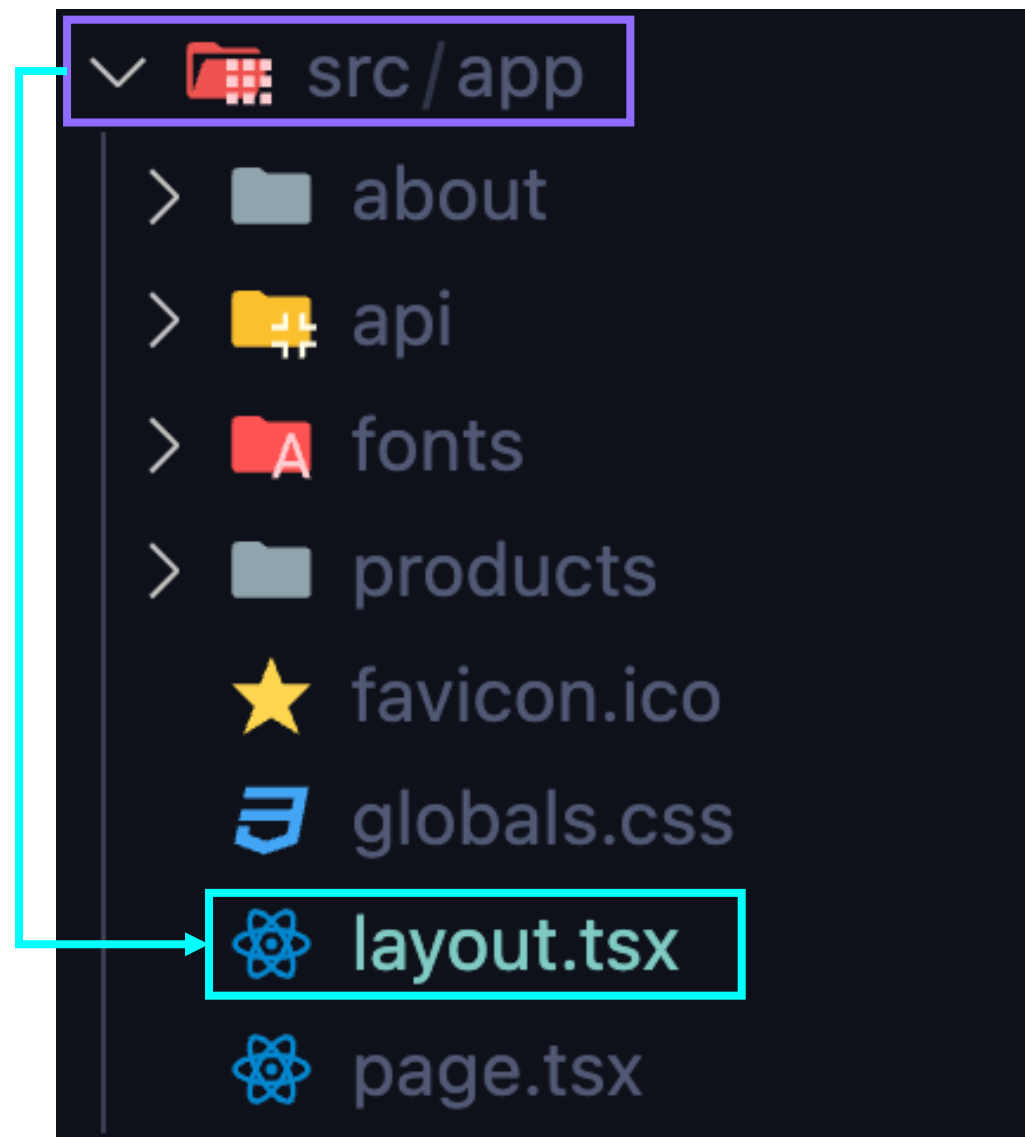
app/layout.tsx

```
import type { Metadata } from "next";
import localFont from "next/font/local";
import "./globals.css";

// ...생략

export default function RootLayout({ children }: Readonly<{ children: React.ReactNode }>){
  return (
    <html lang="en">
      <body className={`\${geistSans.variable} \${geistMono.variable} antialiased`} >
        {children}
      </body>
    </html>
  );
}
```

Layout의 `children` props에는
해당하는 path에 대해 `page.tsx`에서 내보낸 컴포넌트가 들어온다.



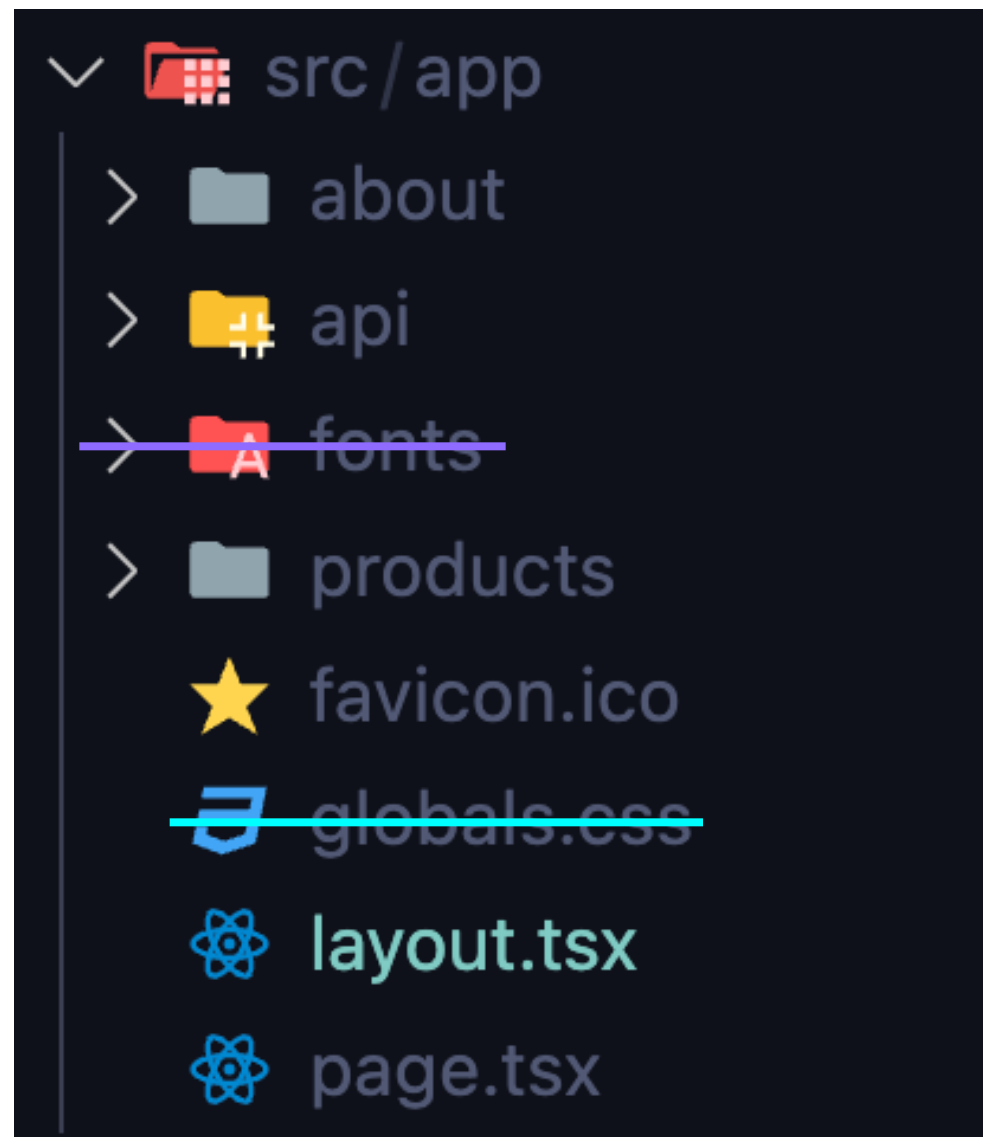
app/layout.tsx

```
import type { Metadata } from "next";
import localFont from "next/font/local";
import "../globals.css";

// ...생략

export default function RootLayout({ children }: Readonly<{ children: React.ReactNode }>){
  return (
    <html lang="en">
      <body className={`\`${geistSans.variable} \`${geistMono.variable} antialiased`}
        {children}
      </body>
    </html>
  );
}
```

Next.js 애플리케이션에서 최상위에 위치한 **Root Layout**은 필수이며,
반드시 **html**과 **body** 태그가 포함되어야 한다.



app/layout.tsx

```
import type { Metadata } from "next";  
import localFont from "next/font/local";  
import "./globals.css";  
  
// ...생략 ← 생략된 코드 모두 삭제  
  
export default function RootLayout({ children }: Readonly<{ children: React.ReactNode }>){  
  return (  
    <html lang="en">  
      <body className=` ${geistSans.variable} ${geistMono.variable} antialiased`>  
        {children}  
      </body>  
    </html>  
  );  
}
```

필요 없는 파일과 코드를 삭제해 보자.

app/layout.tsx

```
export default function RootLayout({ children }: Readonly<{ children: React.ReactNode }>) {
  return (
    <html lang="en">
      <body>
        <h1>Elice with Next.js!</h1>
        {children}
      </body>
    </html>
  );
}
```

그 다음 body 내부에 코드를 추가해 보자.

app/about/page.tsx

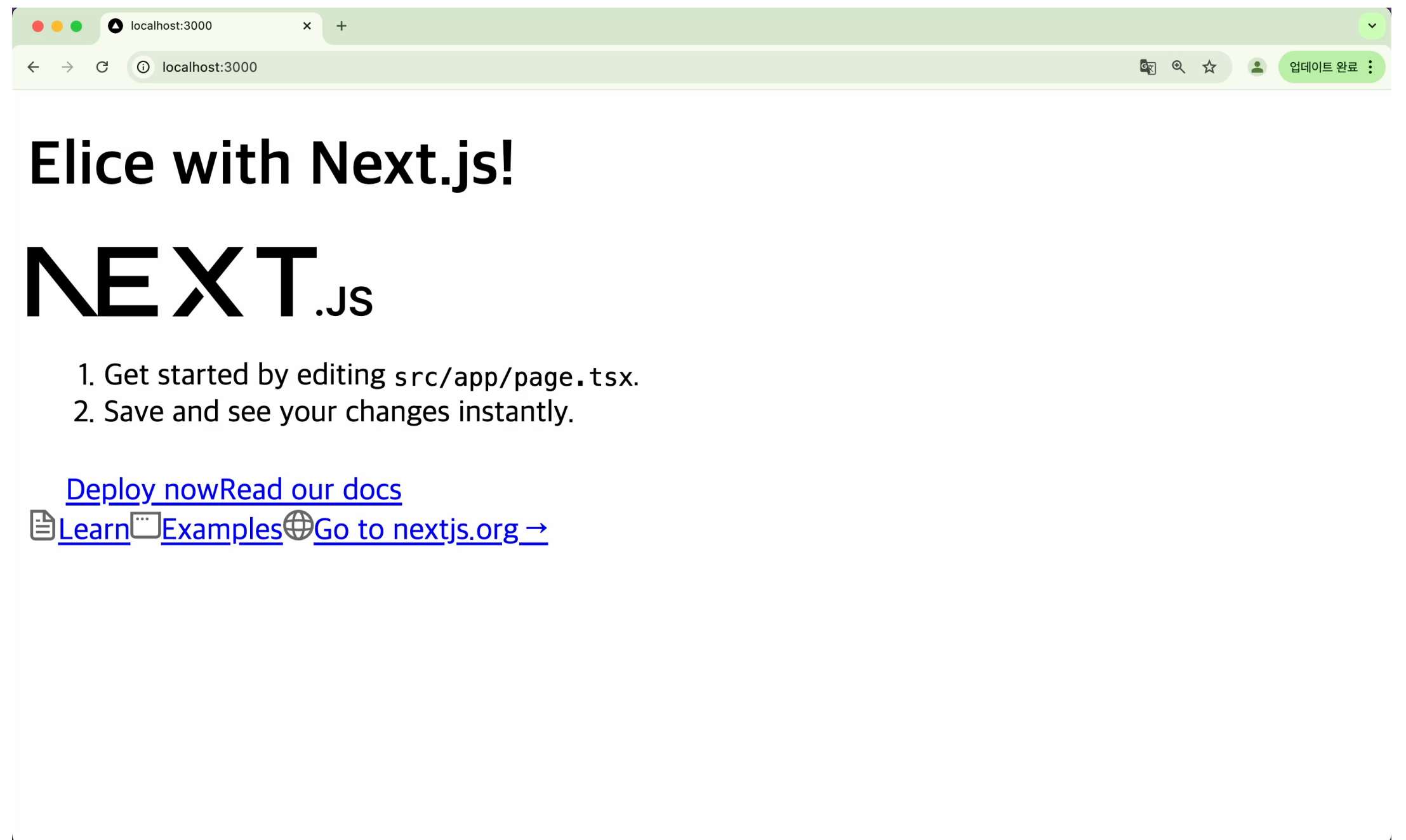
```
export default function () {  
  return <h1>About Page</h1>;  
}
```

app/about/contact/page.tsx

```
export default function () {  
  return <h1>Contact Page</h1>;  
}
```

app/products/list/page.tsx

```
export default function () {  
  return <h1>Products List Page</h1>;  
}
```



↑
Play



- 홈
- 소개
- 연락처
- 제품 리스트

Layout에 Navigation 요소를 추가해 보자.

app/layout.tsx

```
import Link from "next/link";

export default function RootLayout({ children }: Readonly<{ children: React.ReactNode }>) {
  return (
    <html lang="en">
      <body>
        <h1>Elice with Next.js!</h1>
        <ul>
          <li><Link href="/">홈</Link></li>
          <li><Link href="/about">소개</Link></li>
          <li><Link href="/about/contact">연락처</Link></li>
          <li><Link href="/products/list">제품 리스트</Link></li>
        </ul>
        {children}
      </body>
    </html>
  );
}
```

페이지 간 이동에는 next/link에서 제공하는 Link 컴포넌트를 사용한다.

app/layout.tsx

```
import Link from "next/link";

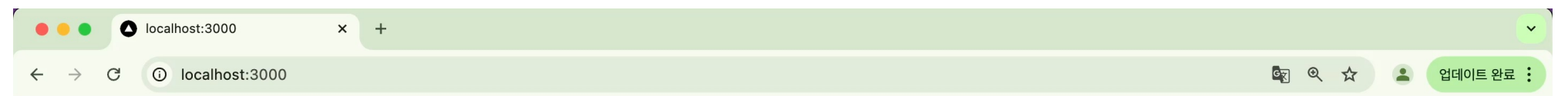
export default function RootLayout({ children }: Readonly<{ children: React.ReactNode }>) {
  return (
    <html lang="en">
      <body>
        <h1>Elice with Next.js!</h1>
        <ul>
          <li><Link href="/">홈</Link></li>
          <li><Link href="/about">소개</Link></li>
          <li><Link href="/about/contact">연락처</Link></li>
          <li><Link href="/products/list">제품 리스트</Link></li>
        </ul>
        {children}
      </body>
    </html>
  );
}
```

이동할 경로는 일반 <a> 태그와 동일하게 href 속성에 작성한다.

app/layout.tsx

```
import Link from "next/link";

export default function RootLayout({ children }: Readonly<{ children: React.ReactNode }>){
  return (
    <html lang="en">
      <body>
        <h1>Elice with Next.js!</h1>
        <ul>
          <li><Link href="/">홈</Link></li>
          <li><Link href="/about">소개</Link></li>
          <li><Link href="/about/contact">연락처</Link></li>
          <li><Link href="/products/list">제품 리스트</Link></li>
        </ul>
        {children}
      </body>
    </html>
  );
}
```



Elice with Next.js!

- [홈](#)
- [소개](#)
- [연락처](#)
- [제품 리스트](#)

NEXT.js

1. Get started by editing src/app/page.tsx.
2. Save and see your changes instantly.

[Deploy now](#)[Read our docs](#)

[Learn](#) [Examples](#) [Go to nextjs.org](#)

↑
Play



왜 Link 컴포넌트가 필요한가?

<a> 태그 vs <Link>컴포넌트 동작 비교

[Full Page Reload]

- 1 **클릭**
브라우저가 서버에 HTML 새로 요청
- 2 **전체 다운로드**
HTML · JS · CSS 모두 재요청
- 3 **React 트리 재빌드**
처음부터 다시 렌더링

- × 화면 깜빡임 (white flash)
- × useState 등 상태 모두 초기화
- × 스크롤 위치 · 입력값 손실
- × 느린 전환 속도

<Link href="/about">

[Client-Side Navigation]

- 1 **클릭 가로채기**
JS가 기본 동작을 중단시킴
- 2 **라우터가 경로 변경**
필요한 컴포넌트만 클라이언트에서 교체
- 3 **Prefetch 활용**
뷰포트에 보이는 경로는 미리 로드

- 깜빡임 없는 부드러운 전환
- React 상태 유지
- 렌더링된 HTML은 그대로 (SEO · 접근성 OK)
- Prefetch로 클릭 즉시 전환

Next.js의 <Link>는 결과물이 <a> 태그이지만, 클릭 동작을 가로채 클라이언트에서 라우팅을 처리한다.



왜 Link 컴포넌트가 필요한가?

12
34 카운트: 0

+1 증가

※ 이 카운터는 layout에 있어요. 페이지를 이동해도 layout이 유지되면 카운트도 유지됩니다.



홈 페이지

위 카운트를 몇 번 올린 뒤, 아래 두 가지 방식으로 About 페이지를 다녀와 보세요.

✗ <a> 태그

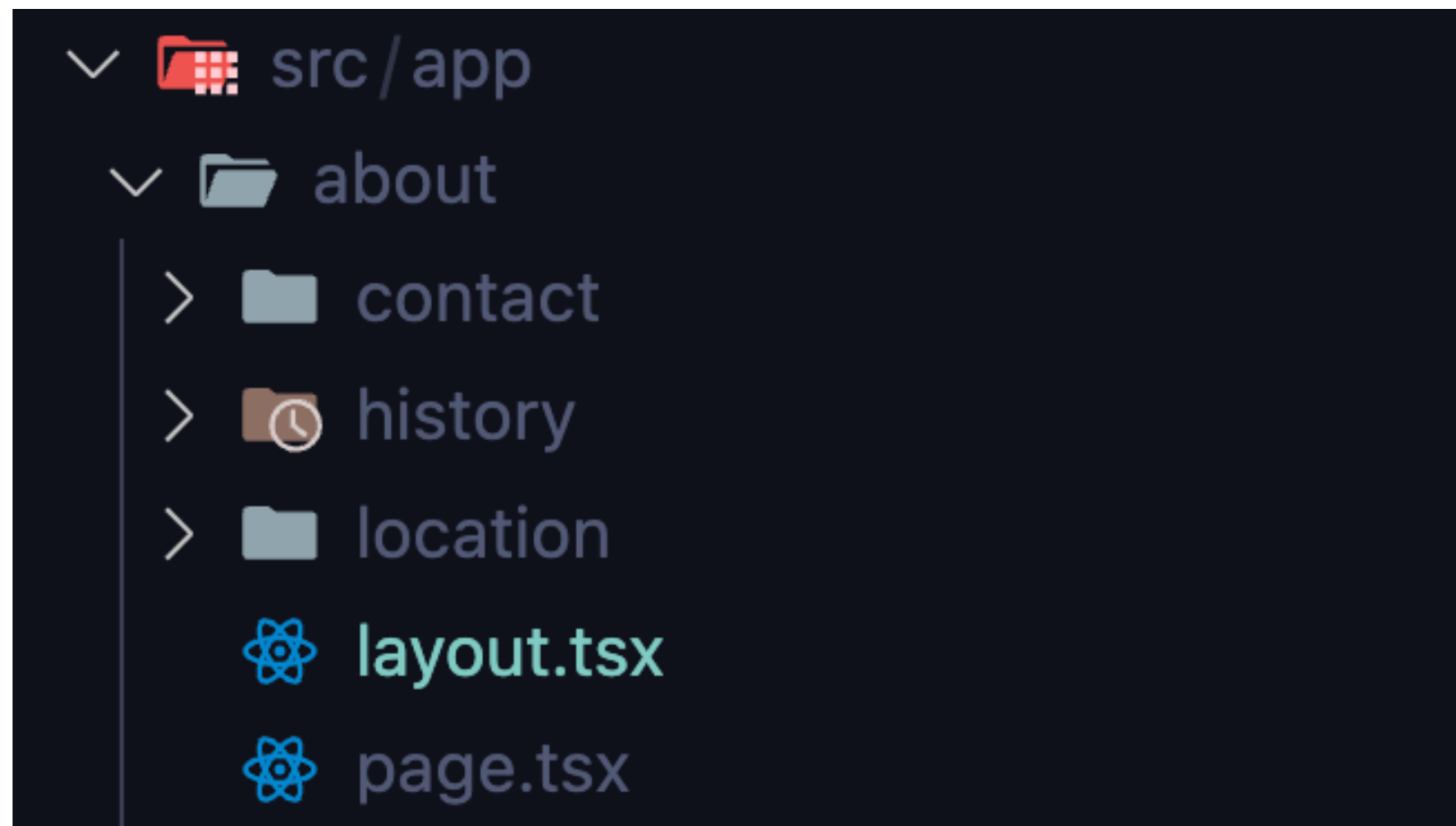
About으로 이동 (a 태그)

클릭 시 화면이 깜빡이고, 돌아오면 카운트가 0으로 초기화됩니다.

✓ <Link> 컴포넌트

About으로 이동 (Link)

클릭 시 깜빡임 없이 부드럽게 전환되고, 돌아와도 카운트가 유지됩니다.



하위 라우트에 layout.tsx를 추가하면, 중첩 Layout을 구현할 수 있다.



중첩 Layout

app/layout.tsx

```
import Link from "next/link";

export default function RootLayout({ children }: Readonly<{ children: React.ReactNode }>) {
  return (
    <html lang="en">
      <body>
        <h1>Elice with Next.js!</h1>
        <ul>
          <li><Link href="/">홈</Link></li>
          <li><Link href="/about">소개</Link></li>
          <li><Link href="/products/list">제품 리스트</Link></li>
        </ul>
        <div style={{ backgroundColor: "lightblue" }}>{children}</div>
      </body>
    </html>
  );
}
```

app/about/layout.tsx

```
import Link from "next/link";

export default function ({ children }: Readonly<{ children: React.ReactNode }>) {
  return (
    <div>
      <h3>About Page</h3>
      <ul>
        <li><Link href="/about">소개 홈</Link></li>
        <li><Link href="/about/contact">연락처</Link></li>
        <li><Link href="/about/history">역사</Link></li>
      </ul>
      <div style={{ backgroundColor: "lightgreen" }}>{children}</div>
    </div>
  );
}
```

하위 라우트의 Layout은 상위 라우트의 Layout의 children으로 들어가게 된다.



중첩 Layout

app/layout.tsx

```
import Link from "next/link";

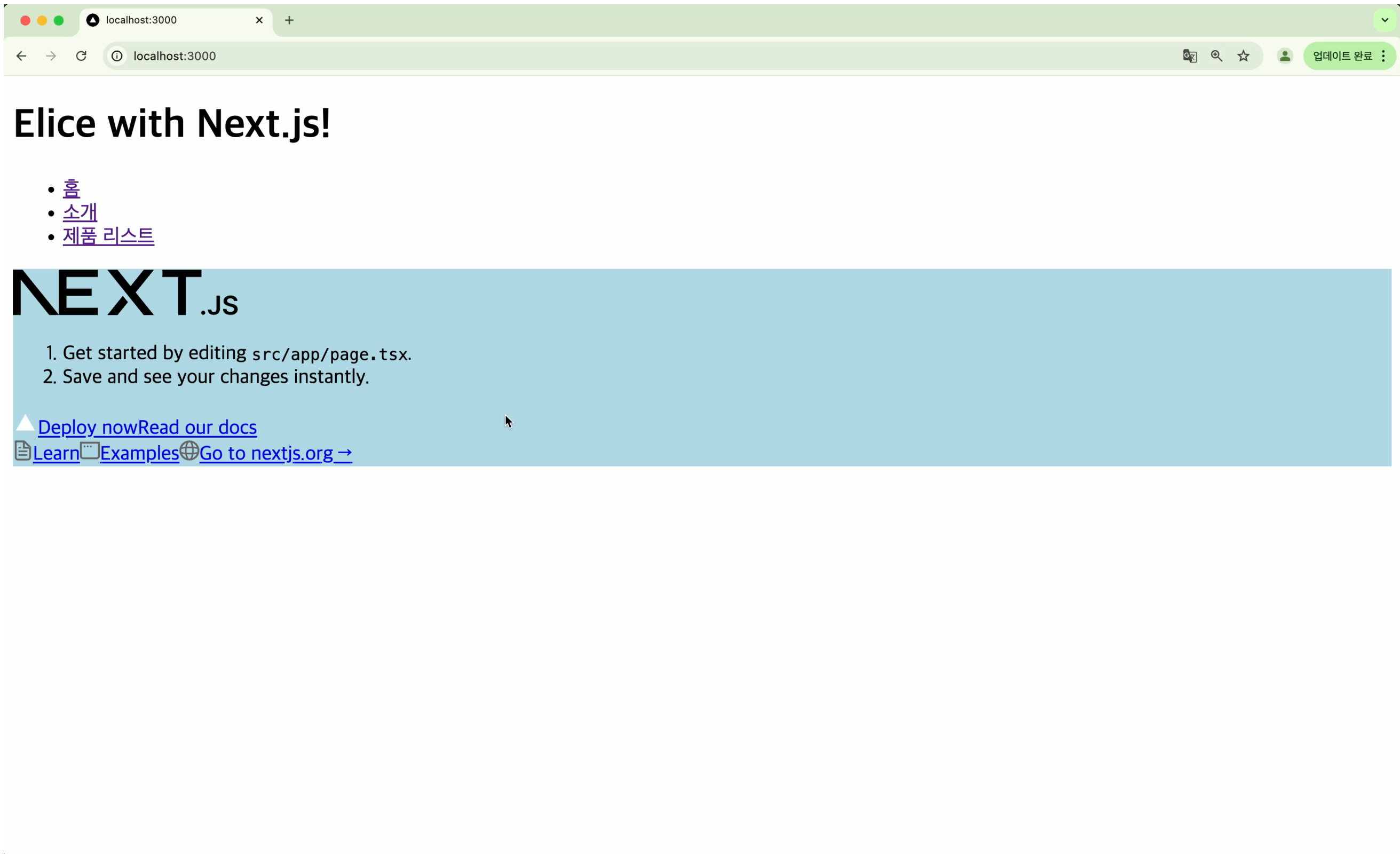
export default function RootLayout({ children }: Readonly<{ children: React.ReactNode }>) {
  return (
    <html lang="en">
      <body>
        <h1>Elice with Next.js!</h1>
        <ul>
          <li><Link href="/">홈</Link></li>
          <li><Link href="/about">소개</Link></li>
          <li><Link href="/products/list">제품 리스트</Link></li>
        </ul>
        <div style={{ backgroundColor: "lightblue" }}>{children}</div>
      </body>
    </html>
  );
}
```

app/about/layout.tsx

```
import Link from "next/link";

export default function ({ children }: Readonly<{ children: React.ReactNode }>) {
  return (
    <div>
      <h3>About Page</h3>
      <ul>
        <li><Link href="/about">소개 홈</Link></li>
        <li><Link href="/about/contact">연락처</Link></li>
        <li><Link href="/about/history">역사</Link></li>
      </ul>
      <div style={{ backgroundColor: "lightgreen" }}>{children}</div>
    </div>
  );
}
```

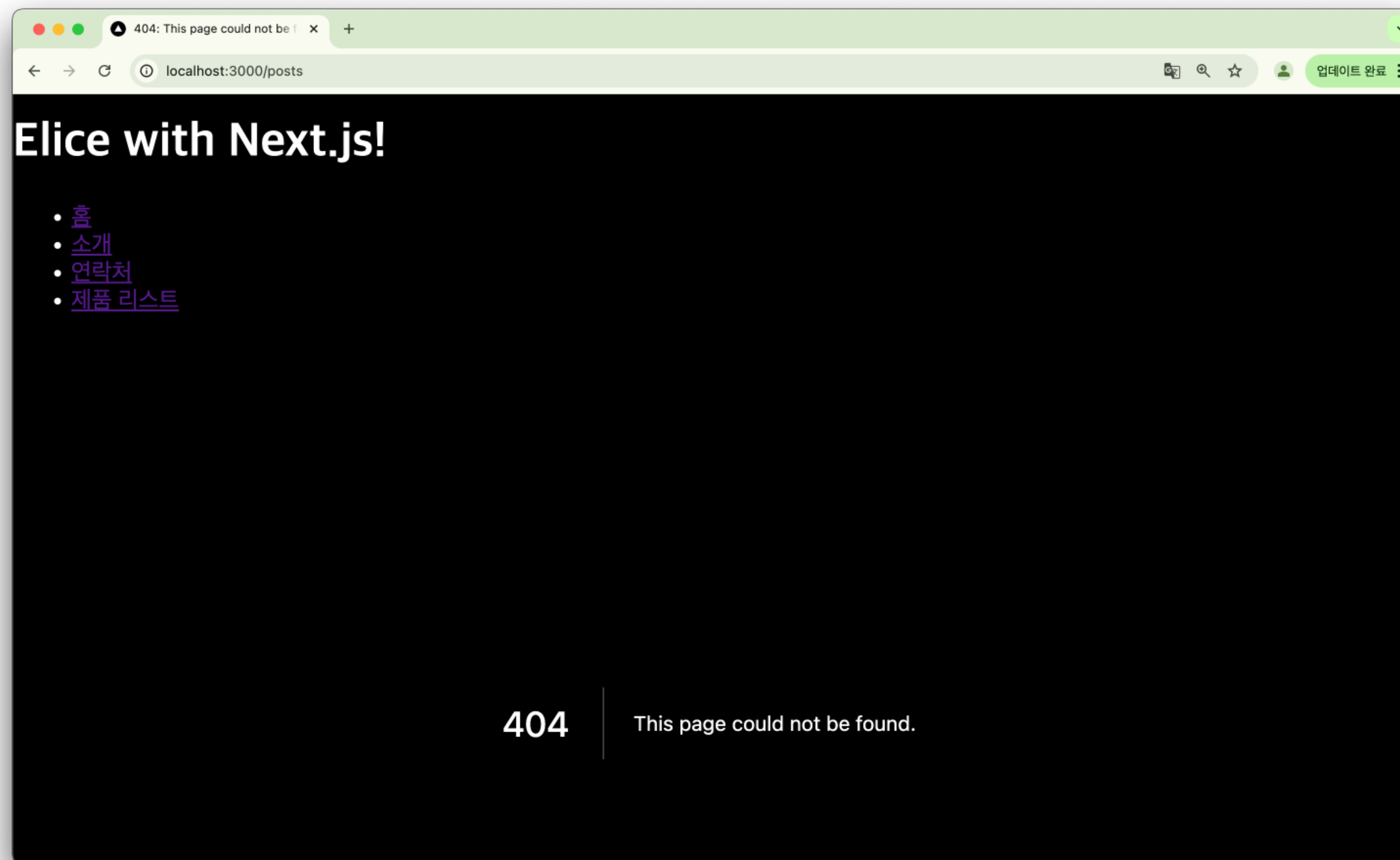
따라서 하위 Layout은 Root Layout과는 달리 html과 body 태그가 들어가지 않아도 된다.



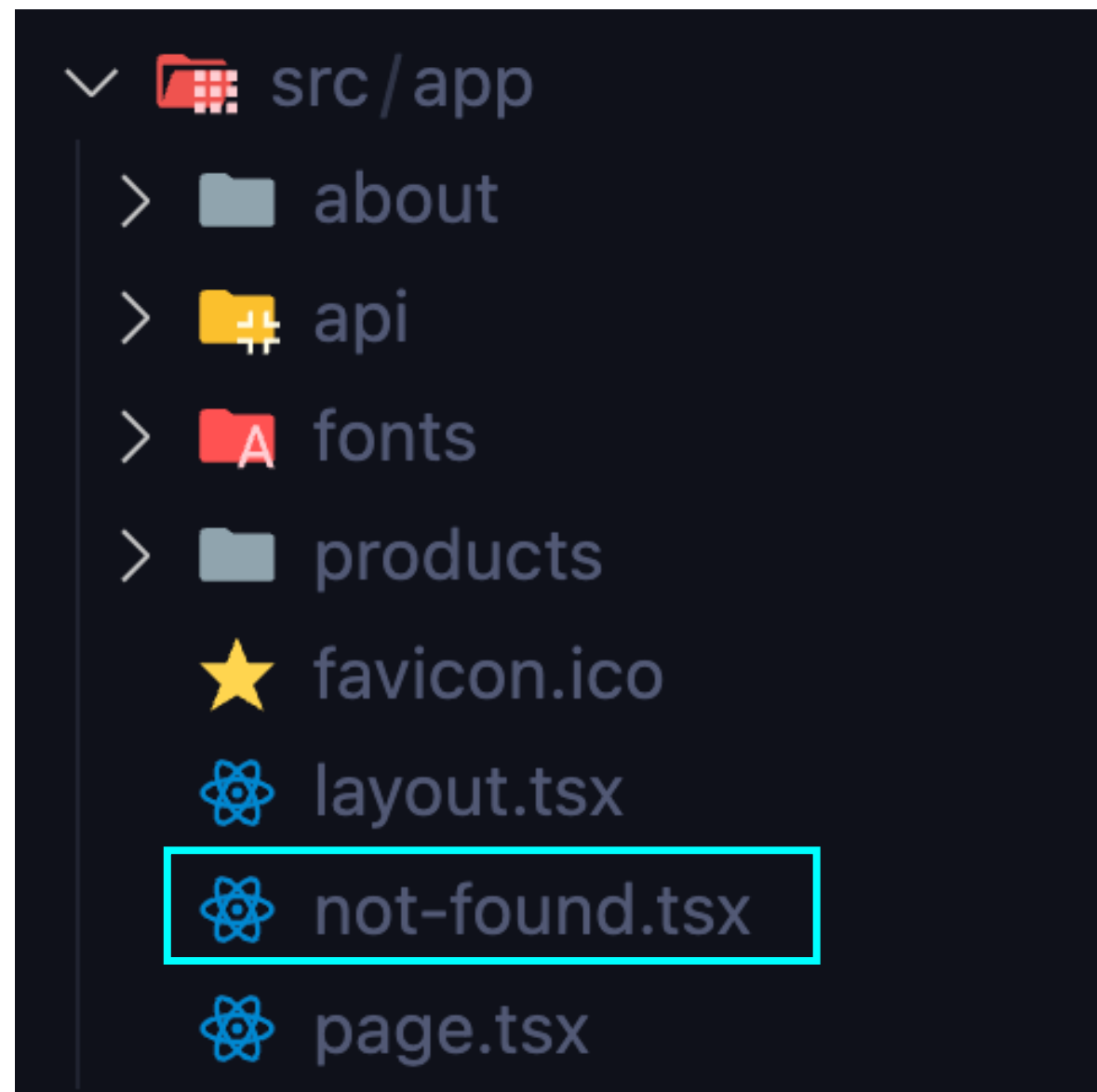
↑
Play



정의하지 않은 경로



Next.js의 라우팅 방식으로 정의하지 않은 경로로 접속하면,
위와 같이 Next.js에서 제공하는 404 페이지가 나타난다.



app/not-found.tsx

```
import Link from "next/link";

export default function () {
  return (
    <>
      <h2>404 Not Found</h2>
      <Link href="/">Return Home</Link>
    </>
  );
}
```

not-found.tsx 파일을 생성하고 컴포넌트를 export하면
정의되지 않은 경로로 접속 시 나타나는 UI를 지정할 수 있다.

app/not-found.tsx

```
import Link from "next/link";

export default function () {
  return (
    <>
      <h2>404 Not Found</h2>
      <Link href="/">Return Home</Link>
    </>
  );
}
```



not-found.tsx 파일을 생성하고 컴포넌트를 export하면
정의되지 않은 경로로 접속 시 나타나는 UI를 지정할 수 있다.



/posts/:id

Path Parameter

→ /posts/1
→ /posts/13
→ /posts/256

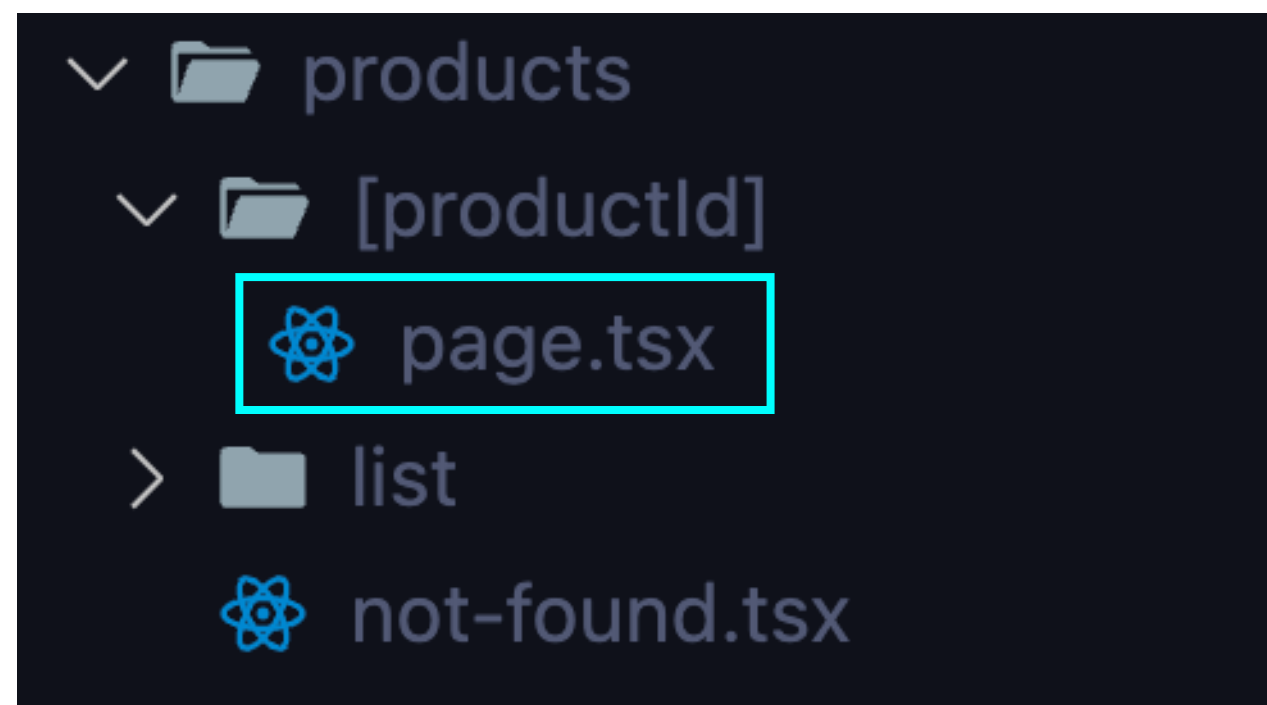
Next.js에서는 Path Parameter를 사용해 동적 라우팅을 구현할 수 있다.



Path Parameter를 사용하는 동적 라우트를 추가하려면
대괄호를 사용하여 `[변수명]` 과 같이 디렉토리 이름을 설정한다.



Path Parameter



app/products/[productId]/page.tsx

```
interface Props {  
  params: {  
    productId: string;  
  };  
}  
  
export default function ({ params: { productId } }: Props) {  
  return <h4>Details of Produce {productId}</h4>;  
}
```

해당 parameter는 page 컴포넌트의 props 중 `params` 프로퍼티로 전달된다.

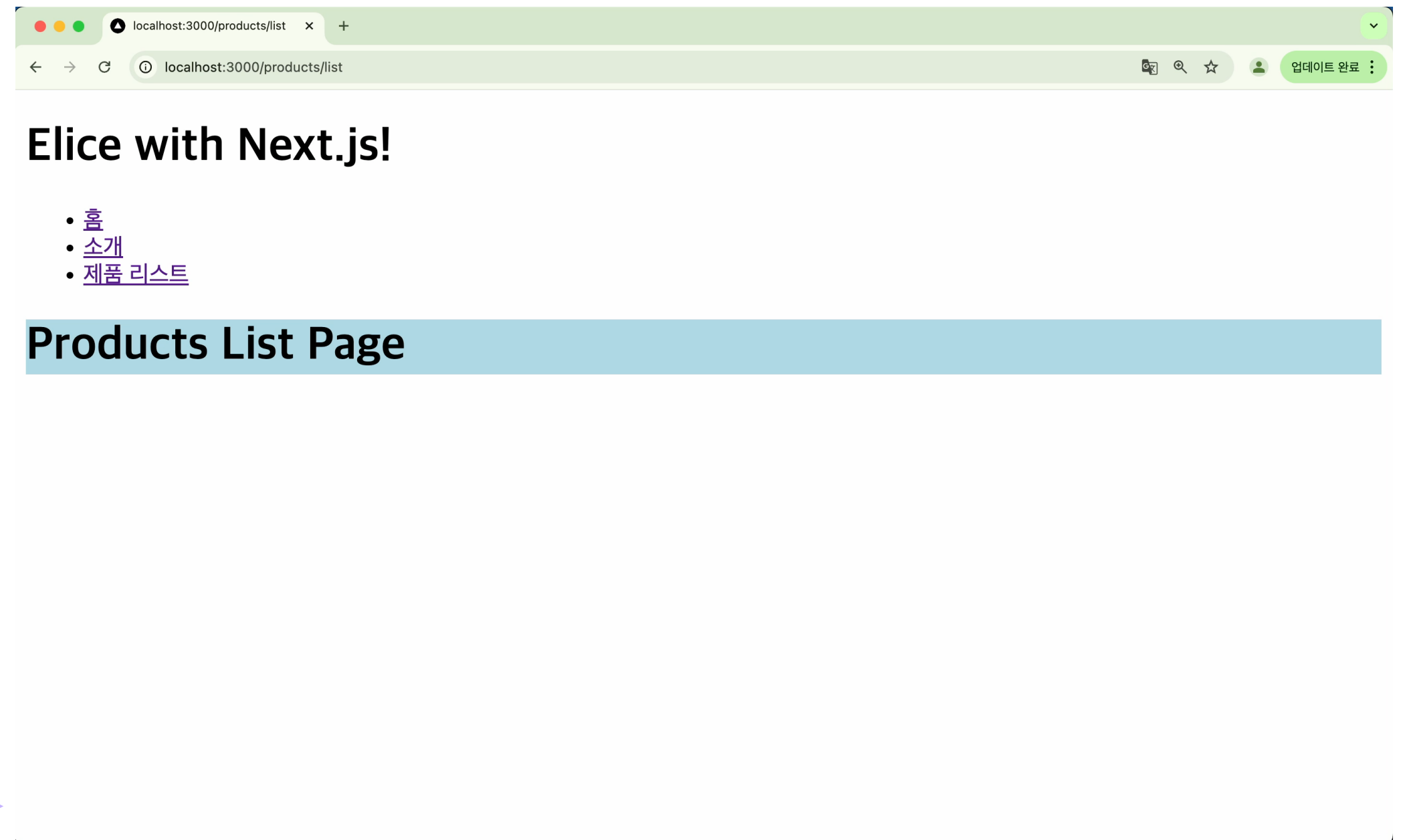


Path Parameter

app/products/[productId]/page.tsx

```
interface Props {  
  params: {  
    productId: string;  
  };  
}  
  
export default function ({ params: { productId } }: Props) {  
  return <h4>Details of Produce {productId}</h4>;  
}
```

Play →



해당 parameter는 page 컴포넌트의 props 중 `params` 프로퍼티로 전달된다.

Next.js 15 변경점 – params는 이제 async

app/products/[productId]/page.tsx **14**

```
interface Props {  
  params: {  
    productId: string;  
  };  
}  
  
export default function ({ params: { productId } }: Props) {  
  return <h4>Details of Produce {productId}</h4>;  
}
```

app/products/[productId]/page.tsx **15+**

```
interface Props {  
  params: Promise<{  
    productId: string;  
  }>;  
}  
  
export default async function ({ params }: Props) {  
  const { productId } = await params;  
  return <h4>Product {productId}</h4>;  
}
```

Next.js 15부터 params는 동기 객체가 아닌 Promise로 사용하도록 도입되었다.

따라서 컴포넌트를 async function으로 선언하고, await으로 꺼내야 한다.

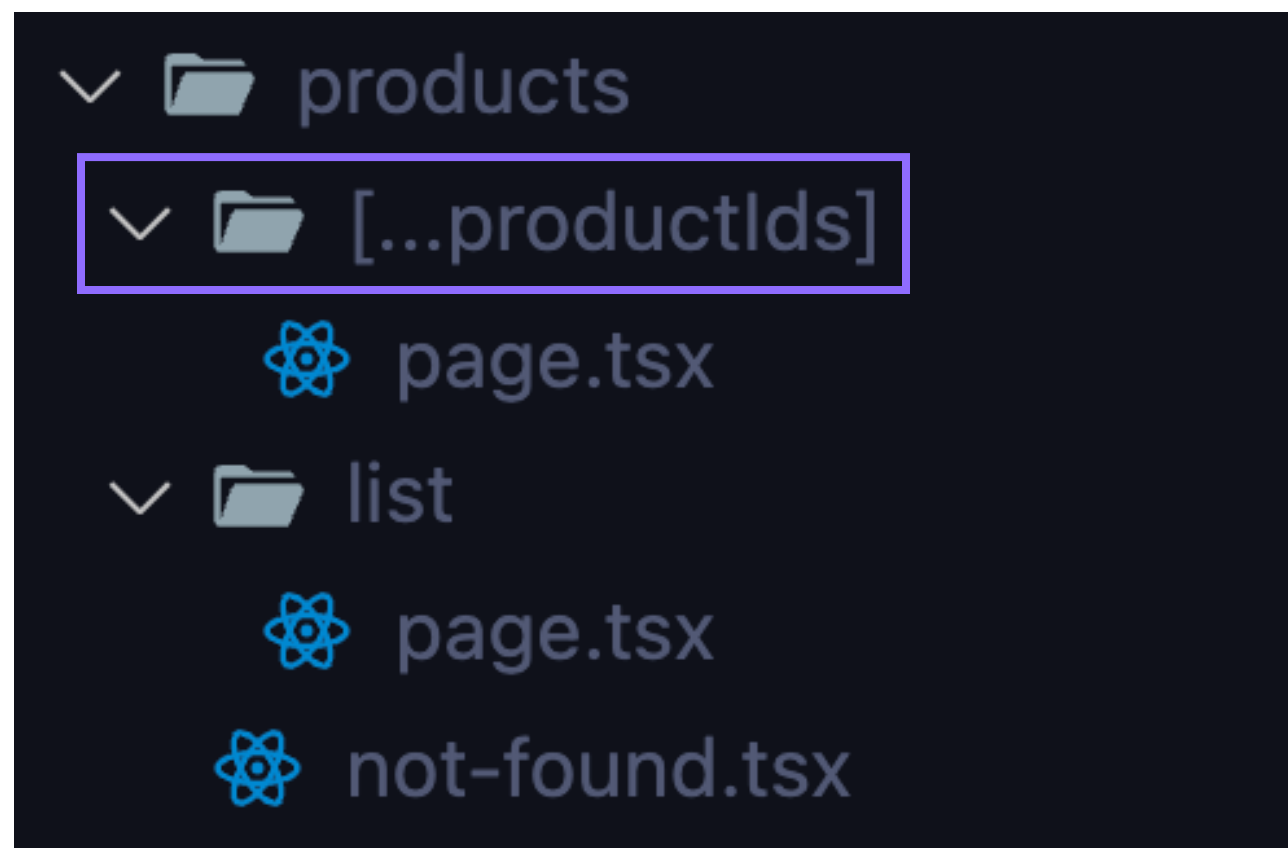
Next.js 15 변경점 – params는 이제 async

app/products/[productId]/page.tsx

```
interface Props {  
  params: Promise<{  
    productId: string;  
  }>;  
}  
  
export default async function ({ params }: Props) {  
  const { productId } = await params;  
  return <h4>Product {productId}</h4>;  
}
```

- params의 타입이 { productId: string; } → Promise<{ productId: string; }>로 변경
- 컴포넌트가 반드시 async function이어야 await 사용 가능

함수 / 파일	params 타입	처리 방식
page.tsx (기본 export)	Promise<{ ... }>	await 필요
layout.tsx (기본 export)	Promise<{ ... }>	await 필요
generateMetadata	Promise<{ ... }>	await 필요
Route Handler (route.ts)	Promise<{ ... }>	await 필요
generateStaticParams	동기 (Promise 아님)	그대로 사용



app/products/[...productIds]/page.tsx

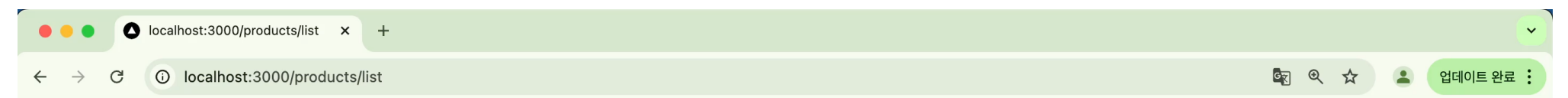
```
interface Props {  
  params: {  
    productIds: string[];  
  };  
}  
  
export default function ({ params: { productIds } }: Props) {  
  return (  
    <>  
      <h4>Details of Products</h4>  
      <ul>{productIds.map((productId, i) => (<li key={i}>{productId}</li>))}</ul>  
    </>  
  );  
}
```

디렉토리 이름을 `[...변수명]`의 형식으로 설정하면,
여러 개의 path parameter를 배열로 받을 수 있다.

app/products/[...productIds]/page.tsx

```
interface Props {  
  params: {  
    productIds: string[];  
  };  
}  
  
export default function ({ params: { productIds } }: Props) {  
  return (  
    <>  
      <h4>Details of Products</h4>  
      <ul>{productIds.map((productId, i) => (<li key={i}>{productId}</li>))}</ul>  
    </>  
  );  
}
```

Play →



Elice with Next.js!

- 홈
- 소개
- 제품 리스트

Products List Page

`/products`와 같이 parameter가 없는 경우는 매칭되지 않는다.



Optional Catch-all



app/products/[...productIds]/page.tsx

```
interface Props {
  params: {
    productIds: string[];
  };
}

export default function ({ params: { productIds } }: Props) {
  return (
    <>
      <h4>Details of Products</h4>
      {productIds ? (
        <ul>{productIds.map((productId, i) => <li key={i}>{productId}</li>)}</ul>
      ) : (
        <h5>No path parameter</h5>
      )}
    </>
  );
}
```

parameter가 없는 경우도 포함하려면, 디렉토리 이름을 `[...변수명]`의 형식으로 설정하면 된다.



Optional Catch-all



↑
Play



Query String

Query String

/posts?**page=2&size=5**

Query String 값 또한 가져올 수 있다.



Query String

app/products/list/page.tsx

```
interface Props {
  params: { /* ... */ };
  searchParams: {
    keyword?: string;
    page: number;
    size: number;
  };
}

export default function ({ searchParams: { keyword, page = 1, size = 10 } }: Props) {
  const products = /* 검색된 제품 리스트 */;

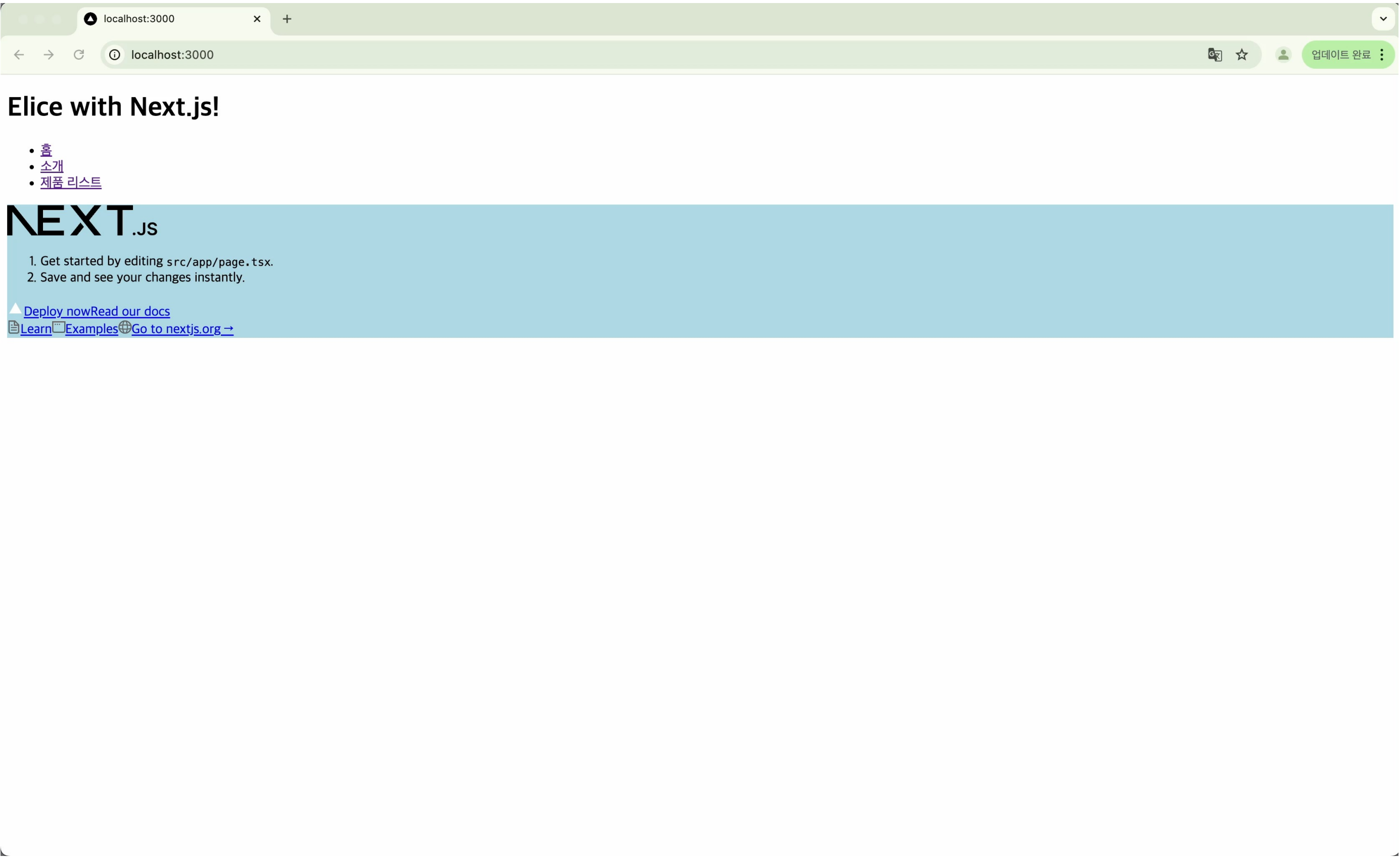
  return (
    <>
      <h2>Product List</h2>
      {keyword && <h4>searched by: {keyword}</h4>}
      <ul>{products.slice(size * (page - 1), size * (page - 1) + size).map((item) => <li>{item}</li>)}</ul>
    </>
  );
}
```

path parameter와 마찬가지로 props로 전달된다.

`searchParams`라는 이름의 프로퍼티로 전달된다.



Query String





Next.js 15 변경점 – searchParams도 async

app/products/list/page.tsx **14**

```
interface Props {
  params: { /* ... */ };
  searchParams: {
    keyword?: string;
    page: number;
    size: number;
  };
}

export default function ({ searchParams: { keyword, page = 1, size = 10 } }: Props) {
  const products = /* 검색된 제품 리스트 */;
  // ...
}
```

app/products/list/page.tsx **15+**

```
interface Props {
  params: Promise<{ /* ... */ }>;
  searchParams: Promise<{
    keyword?: string;
    page: number;
    size: number;
  }>;
}

export default async function ({ searchParams }: Props) {
  const { keyword, page = 1, size = 10 } = await searchParams;
  // ...
}
```

params와 마찬가지로, searchParams 또한 15부터 Promise로 변경되었다.
params와 searchParams 둘 다 async로 바뀐 것이므로 await로 값을 꺼내야 한다.



Next.js 15 변경점 – searchParams도 async

app/products/list/page.tsx 15/16

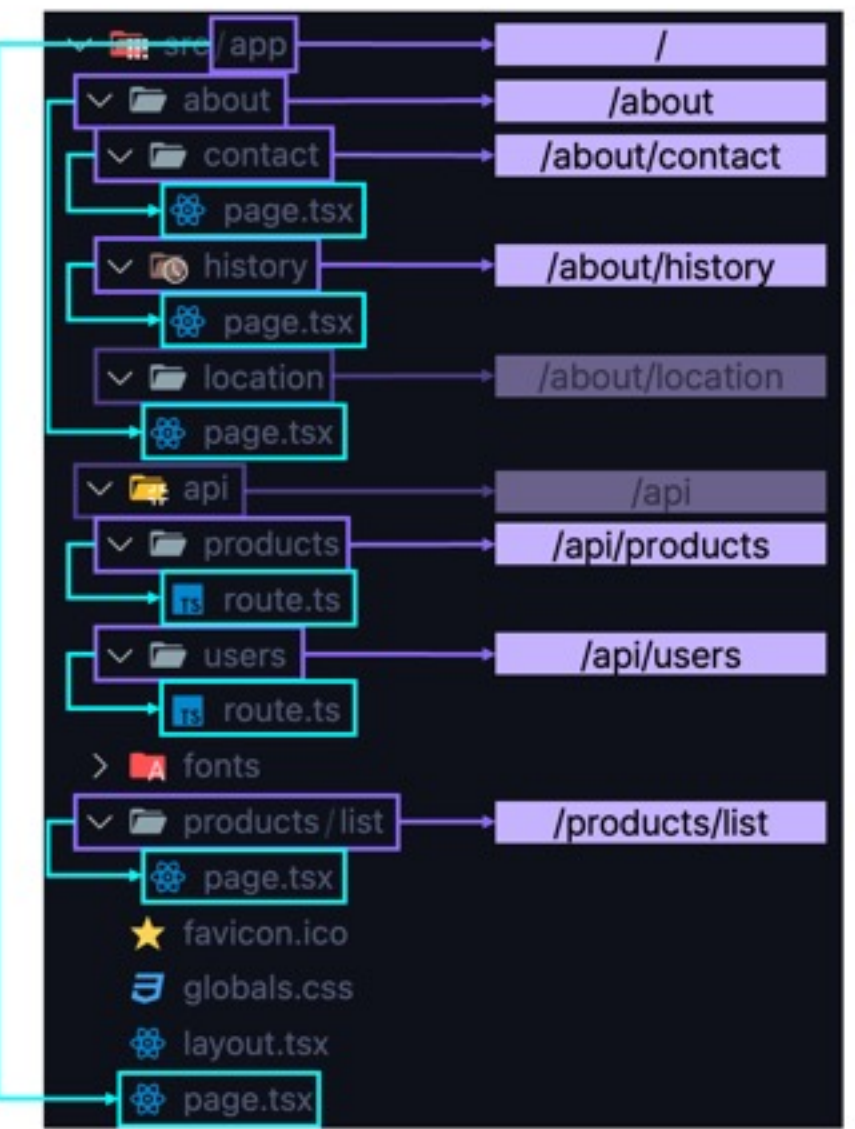
```
export default async function ({ params, searchParams }: Props) {  
  const { productId } = await params;  
  const { keyword } = await searchParams;  
  ...  
}
```

app/products/list/page.tsx 15/16

```
const [{ productId }, { keyword }] = await Promise.all([params, searchParams]);
```

한 컴포넌트에서 둘 다 쓰는 경우 Promise.all로 병렬 처리도 가능하다.

Routing 규칙 3



규칙 3.

디렉토리에 해당하는 path가 실제로 의미를 가지려면, 약속된 파일인 **`page.tsx`** 혹은 **`route.ts`** 파일이 존재해야 한다.

page.tsx (.js, .jsx도 가능)	해당 경로 접속 시 렌더링 할 UI 정의
route.ts (.js도 가능)	해당 경로에 대한 백엔드 API 정의 (추후 학습)

앞서 학습한 Routing 규칙 3에서 path가 실제로 의미를 가지려면, **`page.tsx`** 혹은 **`route.ts`** 파일이 필요하다는 것을 알게 되었다.

→ **`route.ts`** 파일은 어떤 역할을 수행하는 파일일까?



What is Next.js?

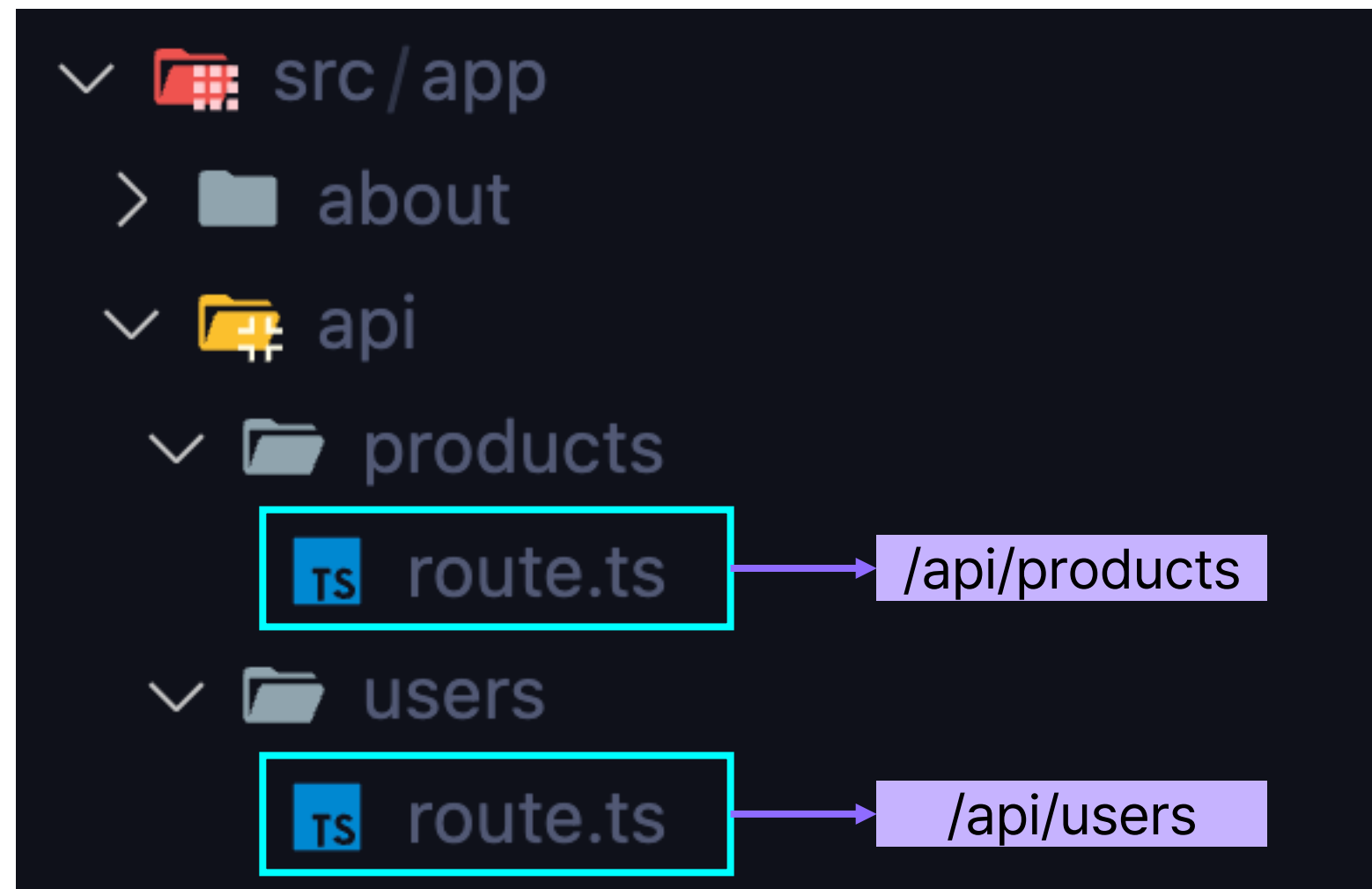
Next.js is a React framework for building **full-stack** web applications. You use React Components to build user interfaces, and Next.js for additional features and optimizations.

Under the hood, Next.js also abstracts and automatically configures tooling needed for React, like bundling, compiling, and more. This allows you to focus on building your application instead of spending time with configuration.

Whether you're an individual developer or part of a larger team, Next.js can help you build interactive, dynamic, and fast React applications.

Next.js는 React 기반의 풀스택 프레임워크이다.

즉, 일반적인 백엔드 API 개발 또한 가능하다.



‘route.ts’ 파일에서 특정 경로에 대한 HTTP Request Handler를 정의할 수 있다.

Next.js에서는 이를 Route Handler라고 한다.



Route Handler

app/api/products/route.ts

```
import { NextRequest, NextResponse } from "next/server";

export async function GET(request: NextRequest) {
  try {
    await dbConnect();
    const products = await Product.find();

    return NextResponse.json(products);
  } catch (e) {
    if (e instanceof Error)
      return NextResponse.json({ error: e.message }, { status: 400 });
  }
}
```

→ GET /api/products

```
export async function POST(request: NextRequest) { /* Something Created */ }
export async function PUT(request: NextRequest) { /* Something Updated */ }
export async function DELETE(request: NextRequest) { /* Something Deleted */ }
```

→ POST /api/products

→ PUT /api/products

→ DELETE /api/products

HTTP Method의 이름을 가진 함수를 export하면,
해당하는 Method에 대한 Request Handler를 등록할 수 있다.



Route Handler

app/api/products/route.ts

```
import { NextRequest, NextResponse } from "next/server";

export async function GET(request: NextRequest) {
  try {
    await dbConnect();
    const products = await Product.find();

    return NextResponse.json(products);
  } catch (e) {
    if (e instanceof Error)
      return NextResponse.json({ error: e.message }, { status: 400 });
  }
}

export async function POST(request: NextRequest) { /* Something Created */ }
export async function PUT(request: NextRequest) { /* Something Updated */ }
export async function DELETE(request: NextRequest) { /* Something Deleted */ }
```

요청에 대한 정보를 얻기 위해 **NextRequest** 타입으로 전달되는 request 매개변수를 사용하고,
응답을 보내기 위해 **NextResponse** 클래스를 활용한다.



Parallel Routes

Middleware

Intercepting Routes

...

Route Groups

지금까지 소개한 기능 외에도, Next.js는 다양한 Routing 심화 기능을 제공한다.

Rendering



Next.js를 사용하는 이유이자 가장 큰 특징이라고 할 수 있는 것은 바로
Next.js만의 최적화 된 Rendering 방식이다.



CSR

Client-Side Rendering

프론트엔드에서
페이지의 동적인 부분
대부분을 처리하는 방식

React는 기본적으로 **Client Side Rendering** 방식을 따른다.

이로 인해 새로고침 없이 동적으로 변화하는 웹을 구현할 수 있고, 더 빠른 웹페이지를 제작할 수 있었다.



CSR

Client-Side Rendering

프론트엔드에서
페이지의 동적인 부분
대부분을 처리하는 방식

그러나 CSR 만을 사용하는 방식에는 한계가 있다.



React의 렌더링 방식과 한계

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="utf-8" />
5     <link rel="icon" href="/favicon.ico" />
6     <meta name="viewport" content="width=device-width, initial-scale=1" />
7     <meta name="theme-color" content="#000000" />
8     <meta
9       name="description"
10      content="Web site created using create-react-app"
11    />
12    <link rel="apple-touch-icon" href="/logo192.png" />
13    <!--
14      manifest.json provides metadata used when your web app is installed on a
15      user's mobile device or desktop. See https://developers.google.com/web/fundamentals/web-app-manifest/
16    -->
17    <link rel="manifest" href="/manifest.json" />
18    <!--
19      Notice the use of ` ` in the tags above.
20      It will be replaced with the URL of the `public` folder during the build.
21      Only files inside the `public` folder can be referenced from the HTML.
22
23      Unlike "/favicon.ico" or "favicon.ico", "/favicon.ico" will
24      work correctly both with client-side routing and a non-root public URL.
25      Learn how to configure a non-root public URL by running `npm run build`.
26    -->
27    <title>React App</title>
28    <script defer src="/static/js/bundle.js"></script></head>
29    <body>
30      <noscript>You need to enable JavaScript to run this app.</noscript>
31      <div id="root"></div>
32
33      This HTML file is a template.
34      If you open it directly in the browser, you will see an empty page.
35
36      You can add webfonts, meta tags, or analytics to this file.
37      The build step will place the bundled scripts into the <body> tag.
38
39      To begin the development, run `npm start` or `yarn start`.
40      To create a production bundle, use `npm run build` or `yarn build`.
41    -->
42  </body>
43 </html>
```

React 페이지의 소스를 확인해 보면,
실제 전달되는 HTML에는 React 컴포넌트에 대한 요소가 전혀 보이지 않음을 알 수 있다.



React의 렌더링 방식과 한계

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="utf-8" />
5     <link rel="icon" href="/favicon.ico" />
6     <meta name="viewport" content="width=device-width, initial-scale=1" />
7     <meta name="theme-color" content="#000000" />
8     <meta
9       name="description"
10      content="Web site created using create-react-app"
11    />
12    <link rel="apple-touch-icon" href="/logo192.png" />
13    <!--
14      manifest.json provides metadata used when your web app is installed on a
15      user's mobile device or desktop. See https://developers.google.com/web/fundamentals/web-app-manifest/
16    -->
17    <link rel="manifest" href="/manifest.json" />
18    <!--
19      Notice the use of ` ` in the tags above.
20      It will be replaced with the URL of the `public` folder during the build.
21      Only files inside the `public` folder can be referenced from the HTML.
22
23      Unlike "/favicon.ico" or "favicon.ico", "/favicon.ico" will
24      work correctly both with client-side routing and a non-root public URL.
25      Learn how to configure a non-root public URL by running `npm run build`.
26    -->
27    <title>React App</title>
28    <script defer src="/static/js/bundle.js"></script></head>
29    <body>
30      <noscript>You need to enable JavaScript to run this app.</noscript>
31      <div id="root"></div>
32
33      This HTML file is a template.
34      If you open it directly in the browser, you will see an empty page.
35
36      You can add webfonts, meta tags, or analytics to this file.
37      The build step will place the bundled scripts into the <body> tag.
38
39      To begin the development, run `npm start` or `yarn start`.
40      To create a production bundle, use `npm run build` or `yarn build`.
41    -->
42  </body>
43 </html>
```

CSR 방식은 이와 같이 비어있는 HTML 소스를 제공하고, 이후 JS가 로드되면서
각 요소가 동적으로 추가되는 방식으로 렌더링된다.



React의 렌더링 방식과 한계

SEO의 어려움

초기 로딩 속도의 문제

JavaScript가 비활성화/오류 발생 시 렌더링 불가

이와 같은 렌더링 방식은 여러 단점이 존재한다.



SSR

Server-Side Rendering

백엔드에서
페이지 대부분의 영역을 미리 처리해서
프론트엔드로 전달하는 방식

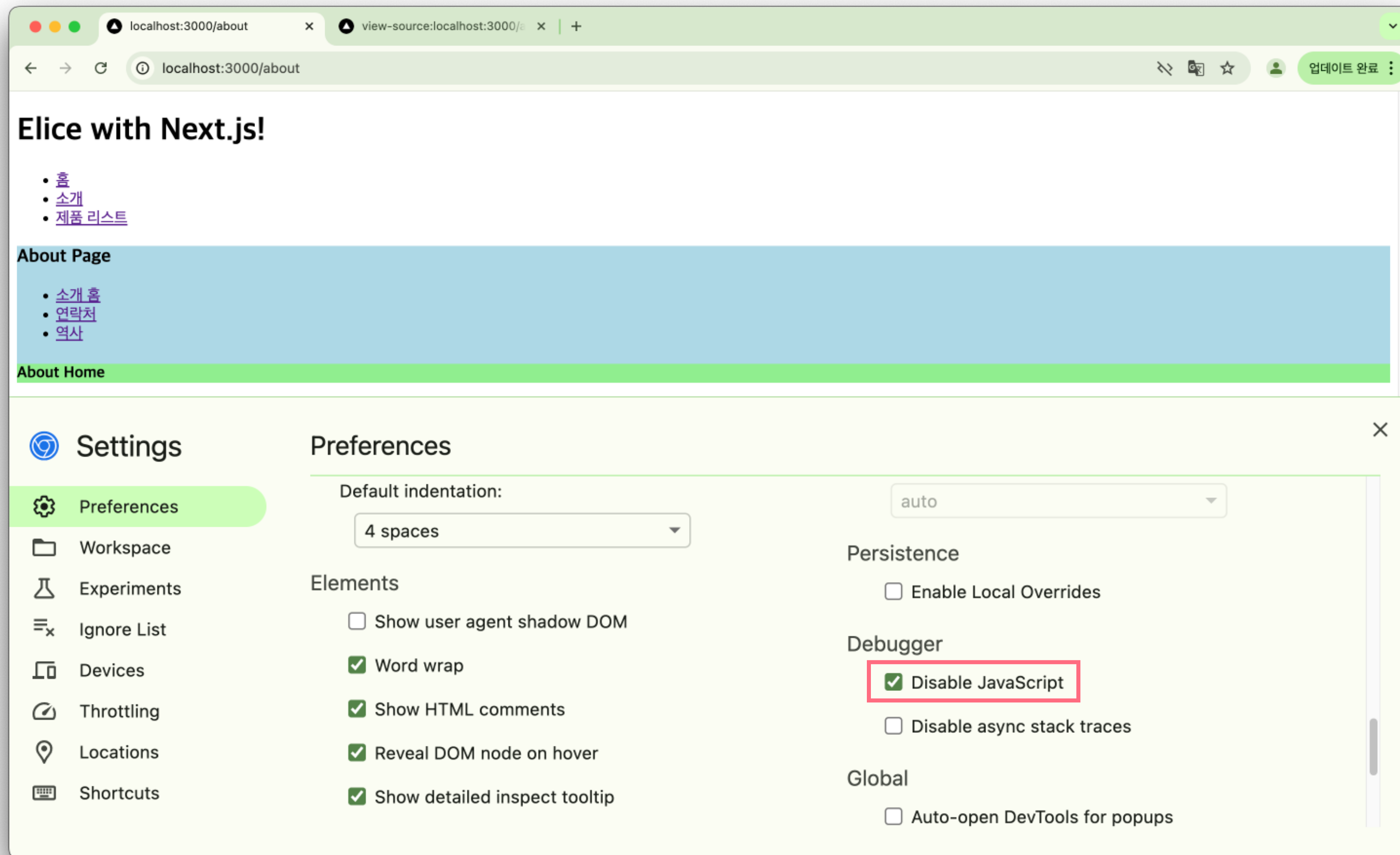
이러한 문제를 해결하기 위해, Next.js는 **Server Side Rendering**을 기본 방식으로 채택했다.



Next.js의 SSR

```
<!DOCTYPE html><html lang="en"><head><meta charset="utf-8"/><meta name="viewport" content="width=device-width, initial-scale=1"/><link rel="preload" as="script"
fetchPriority="low" href="/_next/static/chunks/webpack.js?v=1732471512932"/><script src="/_next/static/chunks/main-app.js?v=1732471512932" async=""></script><script
src="/_next/static/chunks/app-pages-internals.js" async=""></script><script src="/_next/static/chunks/app/about/layout.js" async=""></script><link rel="icon" href="/favicon.ico"
type="image/x-icon" sizes="16x16"/><script src="/_next/static/chunks/polyfills.js" noModule=""></script></head><body><h1>Elice with Next.js!</h1><ul><li><a href="/">홈</a></li>
<li><a href="/about">소개</a></li><li><a href="/products/list">제품 리스트</a></li></ul><div style="background-color:lightblue"><div><h3>About Page</h3><ul><li><a href="/about">소개 홈
</a></li><li><a href="/about/contact">연락처</a></li><li><a href="/about/history">역사</a></li></ul></div><div><h4>About Home</h4></div></div></div>
<script src="/_next/static/chunks/webpack.js?v=1732471512932" async=""></script><script>(self.__next_f=self.__next_f||[]).push([0]);set(self.__next_f.push([2,null])</script>
<script>self.__next_f.push([1,0:D{"name":"r2","env":"Server"}\n])</script><script>self.__next_f.push([1,1:I{"app-pages-
browser)/./node_modules/next/dist/client/components/app-router.js",["app-pages-internals","\static/chunks/app-pages-internals.js"],[""]\n5:I{"app-pages-
browser)/./node_modules/next/dist/client/components/link.js",["app/about/layout","\static/chunks/app/about/layout.js"],[""]\n6:I{"app-pages-
browser)/./node_modules/next/dist/client/components/layout-router.js",["app-pages-internals","\static/chunks/app-pages-internals.js"],[""]\n7:I{"app-pages-
browser)/./node_modules/next/dist/client/components/render-from-template-context.js",["app-pages-internals","\static/chunks/app-pages-internals.js"],[""]\n8:I{"app-pages-
browser)/./node_modules/next/dist/client/components/error-boundary.js",["app-pages-internals","\static/chunks/app-pages-
internals.js"],[""]\n2:D{"name":"","env":"Server"}\n3:D{"name":"__WEBPACK_DEFAULT_EXPORT__","\env":"Server"}\n3:[{"$","\h4",null,{"children":"About
Home"}}\n4:D{"name":"__WEBPACK_DEFAULT_EXPORT__","\env":"Server"}\n4:[{"$","\div",null,{"children":["{"$","\h3",null,{"children":"About Page"}],
["$","\ul",null,{"children":["{"$","\li",null,{"children":["{"$","\h4",null,{"href":"/about","\children":"소개 홈"}]},{"$","\li",null,{"children":"
{"$","\h4",null,{"href":"/about/contact","\children":"연락처"}]}],["$","\li",null,{"children":["{"$","\h4",null,{"href":"/about/history","\children":"역사
"}]}]}],["$","\div",null,{"style":{"backgroundColor":"lightgreen"},"\children":["{"$","\h4",null,{"parallelRouterKey":"children","\segmentPath":
["children","\about","\children","\error":"$undefined","\errorStyles":"$undefined","\errorScripts":"$undefined","\template":["$","\h4",null,
{}],"\templateStyles":"$undefined","\templateScripts":"$undefined","\notFound":"$undefined","\notFoundStyles":"$undefined"}]}]\n8:D{"name":"RootLayout","\env"
:"Server"}\n9:D{"name":"__WEBPACK_DEFAULT_EXPORT__","\env":"Server"}\n9:[{"$","\h2",null,{"children":["404 Not Found"}],["$","\h2",null,
{"href":"/","\children":"Return Home"}],["$","\h2",null,{"children":["$","\h1",null,
{"children":"Elice with Next.js!"}],[""]}]</script><script>self.__next_f.push([1,"$","\ul",null,{"children":["{"$","\li",null,{"children":["{"$","\h4",null,
{"href":"/","\children":"홈"}]},{"$","\li",null,{"children":["{"$","\h4",null,{"href":"/about","\children":"소개"}]}],["$","\li",null,{"children":"
{"$","\h4",null,{"href":"/products/list","\children":"제품 리스트"}]}]}],["$","\div",null,{"style":{"backgroundColor":"lightblue"},"\children":
["$","\h4",null,{"parallelRouterKey":"children","\segmentPath":
["children","\error":"$undefined","\errorStyles":"$undefined","\errorScripts":"$undefined","\template":["$","\h4",null,
{}],"\templateStyles":"$undefined","\templateScripts":"$undefined","\notFound":"$undefined","\notFoundStyles":"
[]}]}}]}]\n2:D{"name":"r0","\env":"Server"}\n2:[{"$","\h1",null,
{"buildId":"development","\assetPrefix":"","urlParts":["","\about"],"\initialTree":["","\children":["about","\children":["__PAGE__",
{}]}],["$undefined","\$undefined","\true"],"\initialSeedData":["","\children":["about","\children":["__PAGE__",{}],["$2","\h3",null,null]},
[null,"$4"],null],["$8",null],["couldBeIntercepted":false,"initialHead":
{"$a","\h1","\globalErrorComponent":"$c","\missingSlots":"$wd"}\n])</script><script>self.__next_f.push([1,"b:[{"$","\meta","\0",
{"name":"viewport","\content":"width=device-width, initial-scale=1"}],["$","\meta","\1",{"charset":"utf-8"}],["$","\link","\2",
{"rel":"icon","\href":"/favicon.ico","\type":"image/x-icon","\sizes":"16x16"}]\n2:null\n])</script></body></html>
```

CSR 방식과 달리, Next.js에서는 컴포넌트의 요소들이 페이지 소스에 포함되어 있는 것을 볼 수 있다.



JS가 비활성화되어도 렌더링에 전혀 문제가 없는 것 또한 확인할 수 있다.



그렇다면 왜 React를 사용하는가?

페이지 전환 시 매번 로드

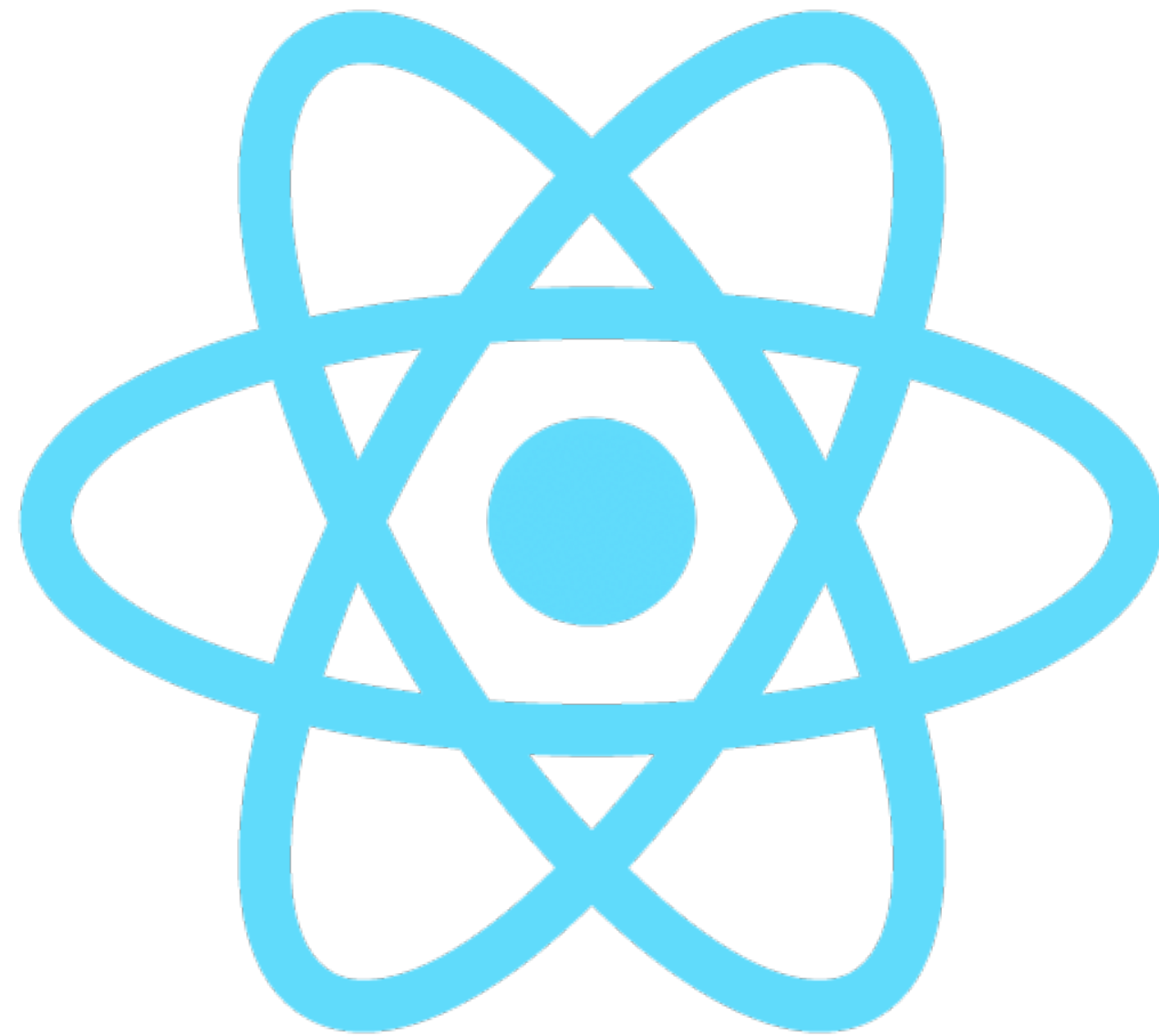
로딩 속도로 인한 UX 저하

이와 같이 Next.js는 CSR의 한계를 극복하기 위해 SSR을 채택했다.

그러나 전통적인 SSR 방식 또한 여러 단점을 가지고 있으며,



그렇다면 왜 React를 사용하는가?



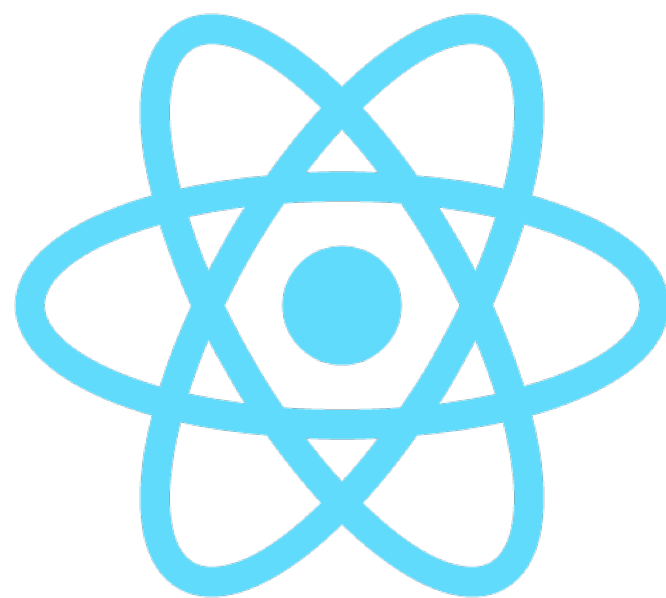
무엇보다 SSR을 사용할 것이라면 React를 사용할 이유가 없어 보인다.



NEXT.JS

하지만 Next.js는 React를 사용한 CSR 및 개발 방식의 이점과,
SSR 방식의 이점을 모두 취할 수 있는 방식을 고안해 내었다.

→ 다음 챕터에서 이에 대해 자세히 학습해 보자.



Server Component와 Client Component



Next.js의 렌더링 방식

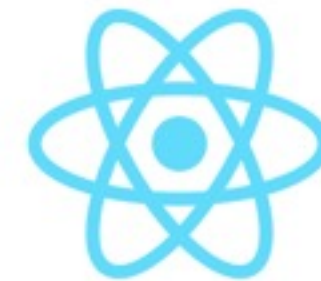
SSR Server-Side Rendering

백엔드에서
페이지 대부분의 영역을 미리 처리해서
프론트엔드로 전달하는 방식



다음 학습 내용 예고

하지만 Next.js는 React를 사용한 CSR 및 개발 방식의 이점과,
SSR 방식의 이점을 모두 취할 수 있는 방식을 고안해 내었다.
→ 다음 챕터에서 이에 대해 자세히 학습해 보자.



Next.js는 SSR 렌더링 방식을 기본으로 하지만,
React 렌더링 방식의 이점 또한 취하는 새로운 방식을 채택했다.



Server Component

Client Component

Next.js 렌더링 방식을 이해하려면

Server Component와 **Client Component**의 개념을 이해해야 한다.



Server Component

모두 Server Component!

app/layout.tsx

```
import Link from "next/link";

export default function RootLayout({ children }: Readonly<{ children: React.ReactNode }>){
  return (
    <html lang="en">
      <body>
        <h1>Elice with Next.js!</h1>
        <ul>
          <li><Link href="/">홈</Link></li>
          <li><Link href="/about">소개</Link></li>
          <li><Link href="/products/list">제품 리스트</Link></li>
        </ul>
        <div style={{ backgroundColor: "lightblue" }}>{children}</div>
      </body>
    </html>
  );
}
```

app/about/layout.tsx

```
export default async function ({ children }: Readonly<{ children: React.ReactNode }>){
  return (
    <>
      <h2>About Elice</h2>
      {children}
    </>
  );
}
```

app/about/characters/page.tsx

```
export default async function () {
  await new Promise((resolve) => setTimeout(resolve, 2000));

  return (
    <>
      <h4>Characters</h4>
      <ul>
        <li>Hello bit</li>
        <li>Cheshire</li>
        <li>Caterpillar</li>
      </ul>
    </>
  );
}
```

Next.js에서 작성하는 모든 컴포넌트는 기본적으로

Server Component이다.



Server Component

app/layout.tsx

```
import Link from "next/link";
```

```
export default function RootLayout({ children }: Readonly<{ children: React.ReactNode }>){
```

서버에서 렌더링

```
  return (  
    <html lang="en">  
      <body>  
        <h1>Elice with Next.js!</h1>  
        <ul>  
          <li><Link href="/">홈</Link></li>  
          <li><Link href="/about">소개</Link></li>  
          <li><Link href="/products/list">제품 리스트</Link></li>  
        </ul>  
        <div style={{ backgroundColor: "lightblue" }}>{children}</div>  
      </body>  
    </html>  
  );  
}
```

app/about/layout.tsx

```
export default async function ({ children }: Readonly<{ children: React.ReactNode }>){  
  return (  
    <>  
      <h2>About Elice</h2>  
      {children}  
    </>  
  );  
}
```

서버에서 렌더링

app/about/characters/page.tsx

```
export default async function () {  
  await new Promise((resolve) => setTimeout(resolve, 2000));
```

서버에서 실행

```
  return (  
    <>  
      <h4>Characters</h4>  
      <ul>  
        <li>Hellobit</li>  
        <li>Cheshire</li>  
        <li>Caterpillar</li>  
      </ul>  
    </>  
  );  
}
```

서버에서 렌더링

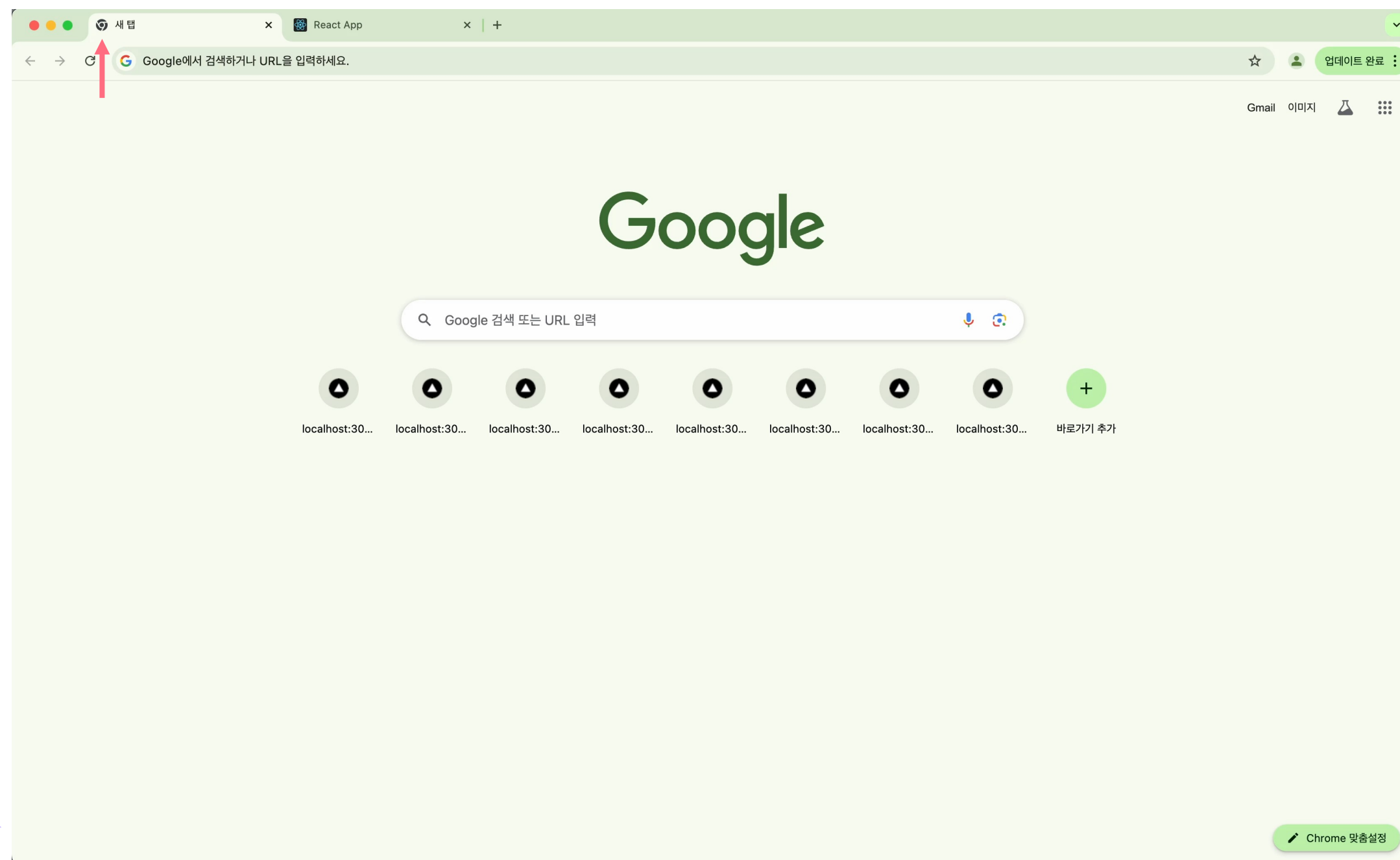
Server Component는

모든 내용이 서버 단에서 실행 및 렌더링 된다.



Server Component

Play →



서버에서의 실행이 완료되어야 페이지가 로드되므로,
접속 시 브라우저 자체에서 로딩이 발생한다. (응답이 완료될 때까지 대기)



Server Component

app/about/characters/page.tsx

async function으로 선언 가능!

```
export default async function() {  
  await new Promise((resolve) => setTimeout(resolve, 2000));  
  
  return (  
    <>  
    <h4>Characters</h4>  
    <ul>  
      <li>Hellobit</li>  
      <li>Cheshire</li>  
      <li>Caterpillar</li>  
    </ul>  
  </>  
);  
}
```

따라서 Server Component는 async function으로 선언 가능하며,



Server Component

app/about/characters/page.tsx

```
import { useState } from "react";

export default async function () {
  // await new Promise((resolve) => setTimeout(resolve, 2000));

  const [characterList, setCharacterList] = useState();

  return (
    <>
      <h4 onClick={() => { alert("Characters"); }}>Characters</h4>
      <ul>
        <li>Hellobit</li>
        <li>Cheshire</li>
        <li>Caterpillar</li>
      </ul>
    </>
  );
}
```

error

error

가장 중요한 특징은 클라이언트 단에서 실행되어야 하는
이벤트 리스너 및 Hook을 사용할 수 없다는 것이다.



Server Component

app/about/characters/page.tsx

```
import { useState } from "react";

export default async function () {
  // await new Promise((resolve) => setTimeout(resolve, 2000));
```

```
const [characterList, setCharacterList] = useState();
```

error

```
return (
```

```
<>
```

```
<h4 onClick={() => { alert("Characters"); }}>Characters</h4>
```

```
<ul>
```

```
<li>Hellobit</li>
```

```
<li>Cheshire</li>
```

```
<li>Caterpillar</li>
```

```
</ul>
```

```
</>
```

```
);
```

```
}
```

error

Error:

× You're importing a component that needs useState. It only works in a Client Component but none of its parents are marked with "use client", so they're Server Components by default.

Learn more: <https://nextjs.org/docs/getting-started/react-essentials>

```
[
  [
    /elice-next/src/app/about/characters/page.tsx:1:1]
1 | import { useState } from "react";
  |
2 |
3 | export default async function () {
4 |   // await new Promise((resolve) => setTimeout(resolve, 2000));
```

× Error: Event handlers cannot be passed to Client Component props.

```
<h4 onClick={function onClick} children=...>
```

If you need interactivity, consider converting part of this to a Client Component.

at stringify (<anonymous>)

digest: "980005539"

이는 결국 Server Component에서는 React의 장점인

이벤트 리스너 관리, Hook, State 관리 등의 기능을 사용하지 못한다는 것을 의미한다.



Client Component

app/about/characters/page.tsx

```
"use client";
```

```
import { useState } from "react";
```

```
export default function () {
```

```
  const [characterList, setCharacterList] = useState();
```

```
  return (
```

```
    <>
```

```
    <h4 onClick={() => { alert("Characters"); }}>Characters</h4>
```

```
    <ul>
```

```
      <li>Hellobit</li>
```

```
      <li>Cheshire</li>
```

```
      <li>Caterpillar</li>
```

```
    </ul>
```

```
  </>
```

```
);
```

```
}
```

```
GET /about/characters 200 in 2043ms  
✓ Compiled in 497ms (554 modules)
```

그래서 이러한 기능이 필요한 컴포넌트를 Client Component로 지정한다.

파일 상단에 "use client"를 추가하여 사용할 수 있다.



Client Component

app/about/characters/page.tsx

```
"use client";
```

```
import { useState } from "react";
```

```
export default function () {
```

```
  const [characterList, setCharacterList] = useState();
```

```
  return (
```

```
    <>
```

```
    <h4 onClick={() => { alert("Characters"); }}>Characters</h4>
```

```
    <ul>
```

```
      <li>Hellobit</li>
```

```
      <li>Cheshire</li>
```

```
      <li>Caterpillar</li>
```

```
    </ul>
```

```
  </>
```

```
);
```

```
}
```

Client Component 역시 SSR을 기반으로 한다. 즉, 요소의 렌더링은 서버에서 수행된다.

("Client" Component라고 해서 CSR이 아님에 주의하자.)



Client Component

app/about/characters/page.tsx

```
"use client";
```

```
import { useState } from "react";
```

```
export default function () {
```

```
  const [characterList, setCharacterList] = useState();
```

```
  return (
```

```
    <>
```

```
    <h4 onClick={() => { alert("Characters"); }}>Characters</h4>
```

```
    <ul>
```

```
      <li>Hellobit</li>
```

```
      <li>Cheshire</li>
```

```
      <li>Caterpillar</li>
```

```
    </ul>
```

```
  </>
```

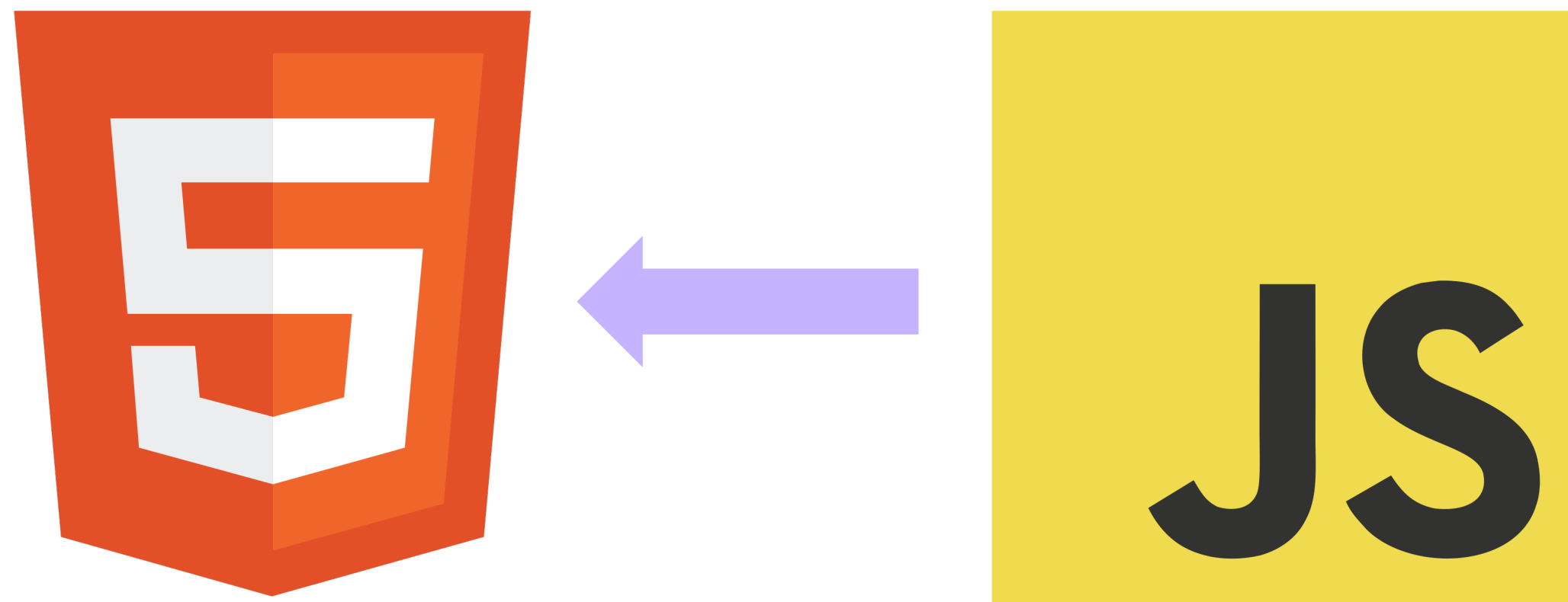
```
);
```

```
}
```

렌더링 된 HTML이 로드된 후, 클라이언트에서는 Hydration이 진행된다.



Client Component



Hydration이란 HTML에 JS 요소가 적용(부착)되는 과정을 의미한다.
브라우저에서 수행되는 자연스러운 동작이며, Next.js/React에만 국한된 개념이 아니다.

렌더링 된 HTML

```
<h4>Characters</h4>  
<ul>  
  <li>Hellobit</li>  
  <li>Cheshire</li>  
  <li>Caterpillar</li>  
</ul>
```

```
onClick={() => { alert("Characters"); }}
```

```
const [characterList, setCharacterList] = useState();
```

따라서 React에서의 hydration은

이미 렌더링 된 요소를 **React 컴포넌트로 만드는 과정**이라고 생각할 수 있다.

→ 이 과정에서 Hook에 대한 설정이 추가되고, 요소에 이벤트 리스너가 등록된다.

Server Component

- Next.js의 기본 값
 - 서버에서 모든 컴포넌트 렌더링
 - 동적인 기능이 필요 없을 때 사용
- Next.js의 여러 최적화 관련 기능의 기반이 됨

Client Component

- “use client”로 지정
 - 서버에서 컴포넌트 렌더링 후 hydration
- 동적인 기능 및 브라우저 API를 사용할 수 있음



언제 사용할까?

What do you need to do?	Server Component	Client Component
Fetch data	✓	✗
Access backend resources (directly)	✓	✗
Keep sensitive information on the server (access tokens, API keys, etc)	✓	✗
Keep large dependencies on the server / Reduce client-side JavaScript	✓	✗
Add interactivity and event listeners (<code>onClick()</code> , <code>onChange()</code> , etc)	✗	✓
Use State and Lifecycle Effects (<code>useState()</code> , <code>useReducer()</code> , <code>useEffect()</code> , etc)	✗	✓
Use browser-only APIs	✗	✓
Use custom hooks that depend on state, effects, or browser-only APIs	✗	✓
Use React Class components ↗	✗	✓

Next.js 공식 문서에는 언제 server component와 client component를 사용하면 좋은지 안내하고 있다.



언제 사용할까?

What do you need to do?	Server Component	Client Component
Fetch data	✓	✗
Access backend resources (directly)	✓	✗
Keep sensitive information on the server (access tokens, API keys, etc)	✓	✗
Keep large dependencies on the server / Reduce client-side JavaScript	✓	✗
Add interactivity and event listeners (<code>onClick()</code> , <code>onChange()</code> , etc)	✗	✓
Use State and Lifecycle Effects (<code>useState()</code> , <code>useReducer()</code> , <code>useEffect()</code> , etc)	✗	✓
Use browser-only APIs	✗	✓
Use custom hooks that depend on state, effects, or browser-only APIs	✗	✓
Use React Class components ↗	✗	✓

client component를 사용하면 좋은 경우는
모두 클라이언트 단에서만 수행 가능한 작업에 대한 설명임을 알 수 있다.



언제 사용할까?

What do you need to do?	Server Component	Client Component
Fetch data	✓	✗
Access backend resources (directly)	✓	✗
Keep sensitive information on the server (access tokens, API keys, etc)	✓	✗
Keep large dependencies on the server / Reduce client-side JavaScript	✓	✗
Add interactivity and event listeners (<code>onClick()</code> , <code>onChange()</code> , etc)	✗	✓
Use State and Lifecycle Effects (<code>useState()</code> , <code>useReducer()</code> , <code>useEffect()</code> , etc)	✗	✓
Use browser-only APIs	✗	✓
Use custom hooks that depend on state, effects, or browser-only APIs	✗	✓
Use React Class components ↗	✗	✓

특이한 것은, data를 fetching하는 작업은 클라이언트에서도 가능함에도 불구하고 server component에서 작업할 것을 권장하고 있다.